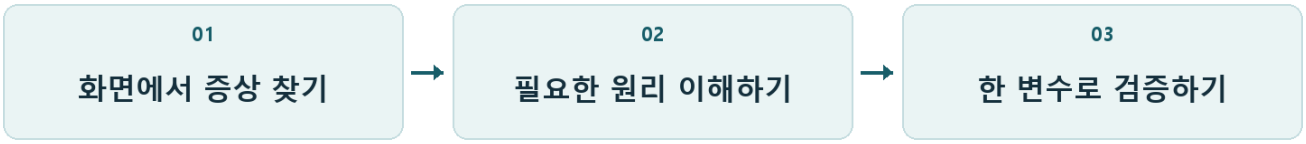
언리얼 엔진 5 렌더링 원리와 최적화

게임 배경 아티스트를 위한 기술 학습 시리즈

게임 배경의 외관과 성능을 원리에 근거해 판단하는 것이 학습의 목적입니다. 재질·조명·에셋 구성의 문제를 진단하고 목표 기기에 맞는 제작 방식을 선택하도록, 26개 주제를 기초부터 연결합니다. 예제는 이 판단 능력을 익히는 실습입니다.



본문에서 필요한 뜻을 먼저 설명합니다

새 용어는 현재 원리를 이해하는 데 필요한 범위에서 먼저 풀어 씁니다. 다음 장의 구현까지 한 문장에 압축하지 않고, 더 깊은 계산과 버전 조건은 보충 주석으로 연결합니다. 문서 끝의 용어집은 배운 내용을 다시 찾는 데 사용합니다.

기준 환경과 실습 상태

UE 5.8 문서와 프로젝트의 UE 5.8.4 데스크톱 경로를 기준으로 합니다. CH01은 실제 쇼윈도 씬과 DX12 캡처를 제공합니다. CH02부터 CH26은 기본 레벨과 따라 할 검증 절차가 준비된 단계입니다. 기능·메뉴·패스 이름은 플랫폼과 설정에 따라 확인합니다.

이미지와 수치 읽기

직접 촬영한 엔진 화면, 직접 제작한 설명도와 외부 참고 이미지를 캡션에서 구분합니다. 외부 이미지에는 저자·원본 링크·이용 조건·편집 여부를 표시합니다. 계산 예시와 실측값을 구분하고, 고해상도 캡처 중의 시간을 일반적인 성능 측정값으로 사용하지 않습니다.

목차

이 문서를 읽는 방법 · 3

렌더링 기초

01 렌더링 파이프라인 · 4
02 Forward와 Deferred · 18
03 GBuffer와 표면 정보 · 20
04 DBuffer와 데칼 · 22
05 Depth와 Early Z · 24
06 Overdraw와 화면 겹침 · 26

머티리얼과 빛

07 PBR과 재질 입력 · 28
08 BRDF와 반사 분포 · 30
09 Fresnel과 시선 각도 · 32
10 Shader와 순열과 WPO · 34
11 Substrate와 재질 레이어 · 36
12 Translucency와 굴절 · 38
13 SSS와 표면 아래 산란 · 40

지오메트리와 텍스처

14 Draw Call과 배칭 · 42
15 Instancing과 반복 배치 · 44
16 LOD와 화면 크기 · 46
17 Nanite와 가상화 지오메트리 · 48
18 Culling과 가시성 · 50
19 Mipmap과 텍스처 샘플링 · 52
20 Texture Streaming과 메모리 예산 · 54

라이팅과 화면 품질

21 Lumen과 동적 간접광 · 56
22 Virtual Shadow Maps와 페이지 캐시 · 58
23 TAA와 프레임 히스토리 · 60
24 TSR과 시간적 업스케일링 · 62

실전 성능 분석

25 CPU와 GPU 병목 · 64
26 GPU Profiler와 전후 검증 · 66

종합 실습 골목 씬의 성능 분석 · 68
용어집 · 69
보충 주석 · 75
출처 목록 · 76

이 문서를 읽는 방법

처음 읽는 순서

CH01의 모델·셰이더·래스터화 설명을 먼저 읽습니다. 이후 각 챕터의 학습 목표로 실무에서 쓸 판단을 확인하고, 원리와 작동 과정을 따라갑니다. 예제 목표는 그 판단 능력을 확인하는 작은 과제입니다.

본문에 남긴 설명

현재 설명의 원인과 결과를 이해하는 데 필요한 용어는 처음 쓰는 자리에서 풀어 설명합니다. 메뉴 이름과 명령은 에디터에서 찾을 수 있도록 영문을 함께 둡니다. 설명을 읽고 장면에서 무엇을 볼지 연결합니다.

깊게 읽을 때

수식, 구현 조건과 버전 차이를 더 읽고 싶다면 보충 주석을 참고합니다. 한 주제를 공부하는 데 다른 주제의 원리가 필요하면 챕터 끝의 관련 개념 링크로 이동합니다.

모르는 용어를 다시 만났을 때

용어집에는 짧은 뜻과 설명한 챕터를 적었습니다. 정의만 외우기보다 그 챕터의 관찰 사례로 돌아가 값과 화면 결과가 어떻게 연결되는지 확인합니다.

같은 조건으로 비교하기

카메라·출력 크기·조명·노출·품질 설정을 기록합니다. 비교 대상 외의 조건을 유지하고 한 변수만 바꿉니다. 정지 화면과 이동 영상을 함께 확인하고, 비용은 같은 길이의 반복 실행으로 측정합니다.

시간 단위 읽기

FPS는 초당 표시할 프레임 수입니다. ms는 밀리초로 1초의 천분의 1이며, 이 문서에서는 주로 한 작업이나 프레임에 걸린 시간을 표시합니다. 작업 시간이 짧아지는 것과 최종 FPS가 오르는 것은 병목에 따라 다를 수 있습니다.

설정과 실제 비용 구분하기

품질 설정 묶음은 Scalability라고 부릅니다. 설정값, 셰이더 통계와 실제 실행 시간은 서로 다른 정보를 줍니다. 목표 기기에서 원하는 외관과 프레임 시간을 함께 확인합니다.

외부 이미지를 찾아볼 곳

좌표 변환 6쪽 · 투영 비교 7쪽 · GBuffer 21쪽 · 재질 구 29쪽 · Fresnel 33쪽 · LOD 47쪽 · Nanite 49쪽 · 밉맵 53쪽. 각 이미지 아래에서 원본과 이용 조건을 열 수 있습니다.

01 렌더링 파이프라인

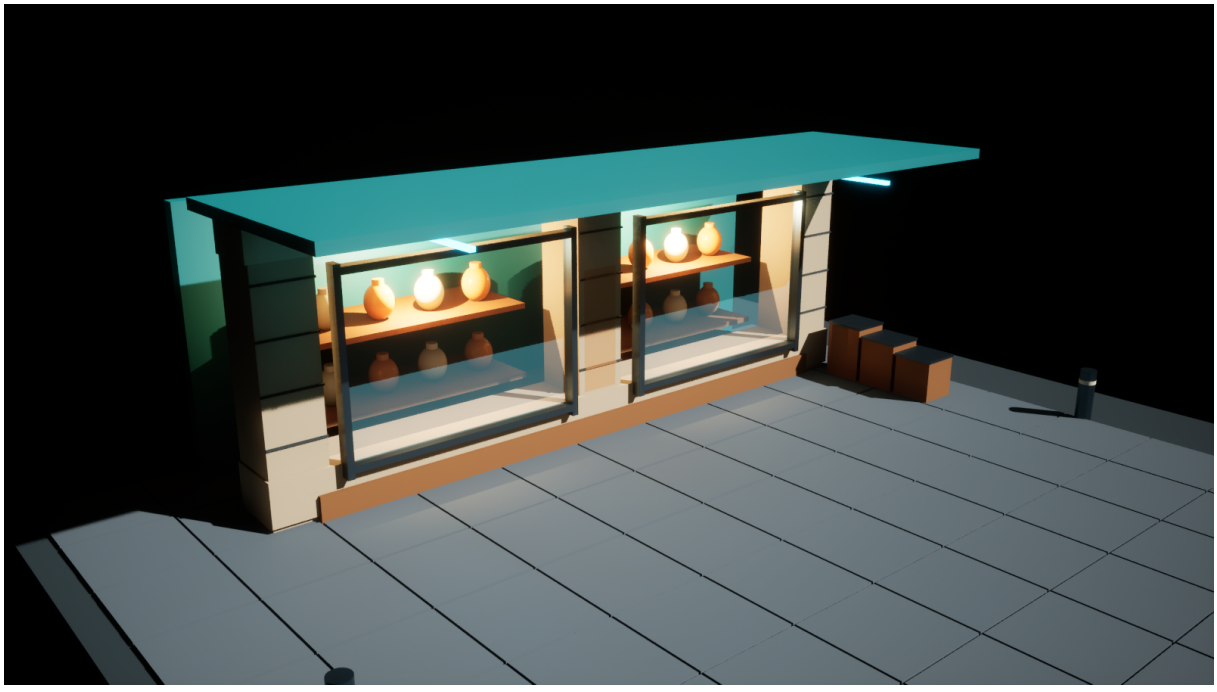
챕터 학습 목표

화면이 만들어지는 과정을 이해해 시각적 오류와 성능 문제를 담당하는 단계의 후보를 좁힙니다. 재질·조명·후처리 중 무엇을 수정할지 판단하고, 성능 측정 도구와 엔진 담당자에게 문제를 구체적으로 전달할 수 있어야 합니다.

> **예제 학습 목표**
>
> 쇼윈도 씬의 표면·깊이 진단 화면을 읽고 유리·데칼·Bloom을 하나씩 바꿔, 변경한 요소와 최종 화면의 관계를 확인합니다.

이 예제의 구성

작은 상점에 벽·프레임·진열대, 뒤가 비치는 유리, 벽에 투영하는 표식인 데칼과 밝게 빛나는 바를 배치했습니다. 여러 요소가 최종 화면에 기여하는 방식을 관찰하는 장면입니다. UE에서 레벨에 배치하는 오브젝트 단위를 Actor라고 합니다.



직접 제작·엔진 캡처. CH01 골목의 Lit 화면. 동일 카메라로 진단 화면과 A/B 결과를 비교합니다.

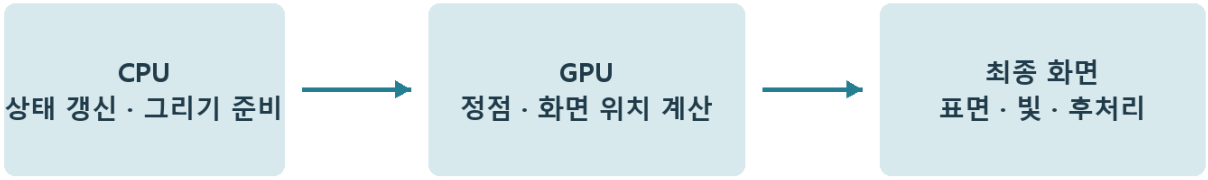
실습 레벨

/Game/RenderingLab/Chapters/Part01_RenderingBasics/CH01_RenderingPipeline/Maps/L_CH01_RenderingPipeline

읽는 순서

먼저 다음 두 쪽에서 모델·셰이더·렌더링 단계의 뜻을 읽습니다. 이후 같은 구도의 표면·깊이 화면과 한 요소만 바꾼 결과를 비교합니다.

모델이 화면에 보이기 위한 기초



모델의 삼각형 정보와 재질 계산을 화면 결과로 연결합니다.

직접 제작 개념도. CPU의 준비와 GPU의 계산이 화면 결과에 기여합니다. 두 장치는 작업을 겹쳐 처리할 수 있습니다.

CPU와 GPU

CPU는 게임 규칙, 오브젝트 상태와 여러 작업의 준비를 처리하는 장치입니다. GPU는 많은 정점과 화면 위치의 계산을 병렬로 처리하도록 설계된 장치입니다. 화면 계산을 맡은 엔진 부분을 렌더러라고 부릅니다. 둘이 작업을 나누므로 GPU가 빨라도 CPU의 준비가 늦으면 프레임 진행이 제한될 수 있습니다.

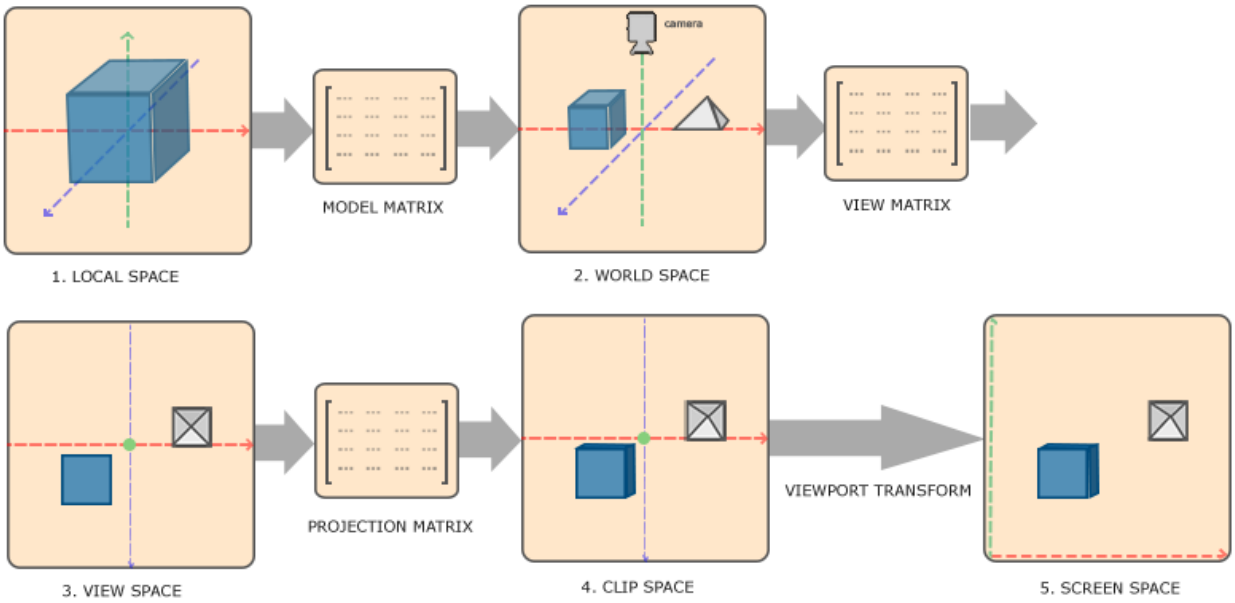
지오메트리와 메시

지오메트리는 모델의 위치와 모양을 정하는 기하 정보입니다. 이 실습의 메시, 즉 3D 표면 형상은 주로 삼각형으로 구성됩니다. 정점은 삼각형의 꼭짓점입니다. 바닥의 포장석을 구분하는 틈은 이 예제에서는 실제 형상으로 만들었으므로 깊이와 삼각형 표시에서도 확인할 수 있습니다.

셰이더의 역할

셰이더는 GPU에서 실행하는 계산 프로그램입니다. 정점 셰이더는 꼭짓점의 위치 등을, 픽셀 셰이더는 화면에서 표면을 표현할 값 등을 처리합니다. 머티리얼 그래프는 이런 표면 반응을 노드로 제작하는 화면이며, 그래프 전체가 하나의 픽셀 셰이더와 항상 일대일로 대응하는 것은 아닙니다.

정점 위치와 표면 정보가 전달되는 방식



외부 참고 그림 · 모델의 로컬 공간부터 월드·카메라·클립·화면 공간까지 이어지는 좌표 변환입니다.

읽을 곳 1→2에서는 큐브를 장면에 배치하고, 2→3에서는 카메라를 기준으로 장면을 해석합니다. 4→5에서는 투영한 결과가 화면 영역에 대응합니다. UE와 OpenGL의 축·깊이 규약은 같지 않지만 좌표 기준을 바꾸는 목적은 같습니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · CC BY 4.0 · 출처 [37]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

모델 좌표에서 월드 좌표로

그림 1의 꼭짓점은 모델 내부 기준으로 표현됩니다. 그림 2에서는 그 모델을 장면의 어느 위치·크기·방향으로 배치했는지 반영합니다. 예를 들어 회전과 확대 없이 모델 원점을 월드 X=10에 놓으면, 로컬 X=1인 꼭짓점은 월드 X=11이 됩니다.

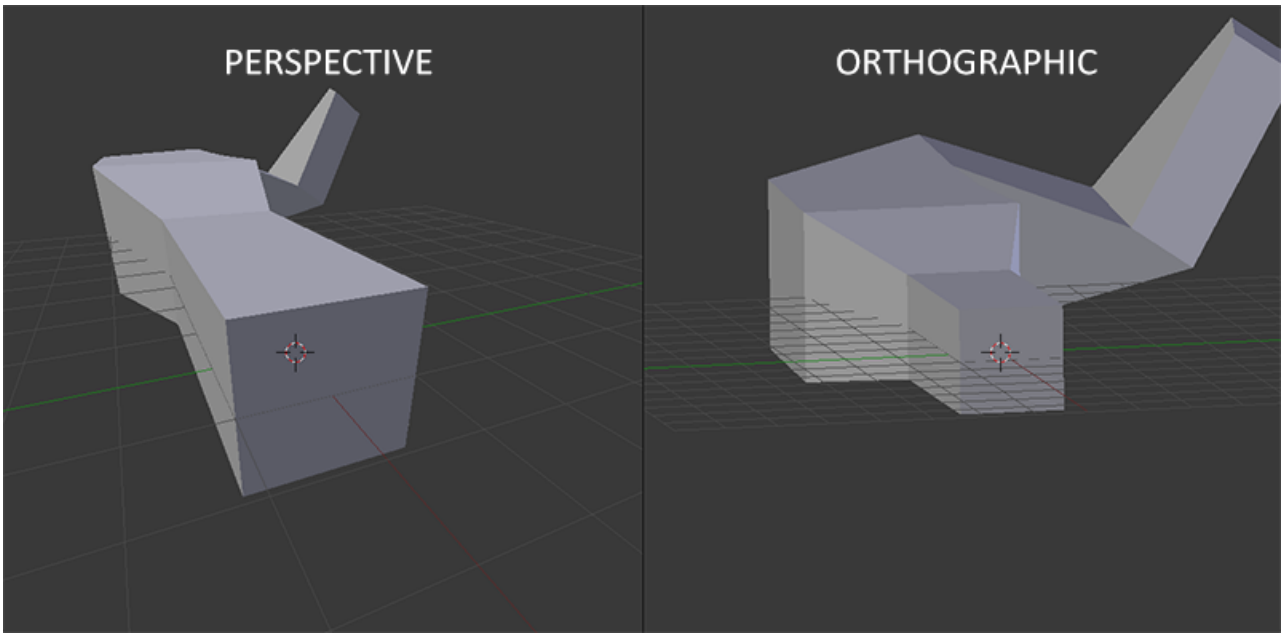
카메라를 기준으로 다시 해석하기

그림 3은 물체가 카메라 기준으로 어디에 있는지 표현합니다. 카메라의 위치와 방향을 바꾸면 같은 월드 위치도 다른 카메라 좌표로 표현됩니다. 물체가 실제로 다른 곳으로 이동해야만 좌표값이 바뀌는 것은 아닙니다.

화면 영역으로 대응시키기

그림 4의 클립 공간에서는 투영과 잘라낼 범위를 처리하고, 원근 나눗셈을 거친 뒤 그림 5의 화면 영역에 대응시킵니다. 여기서 행렬은 위치·방향·크기 등을 수의 묶음으로 계산하는 도구입니다. 숫자의 구체적인 규약은 그래픽 API와 엔진 구현에 따라 확인합니다.

원근 투영과 직교 투영의 차이



외부 비교 이미지 · 같은 형상의 투영 방식 비교입니다. 왼쪽 Perspective(원근), 오른쪽 Orthographic(직교)입니다.

읽을 곳 왼쪽은 뒤로 갈수록 크기와 간격이 작아 보입니다. 오른쪽은 깊이에 따른 크기 변화가 없어 평행한 방향을 읽기 쉽습니다. 투영 방식의 비교이며 UE의 캡처가 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · CC BY 4.0 · 출처 [37]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

원근 투영은 깊이에 따른 크기를 보여 줍니다

일반적인 게임 카메라는 멀리 있는 같은 크기 물체가 작아 보이도록 원근을 반영합니다. 그림 왼쪽에서 앞쪽과 뒤쪽 부분의 크기·방향이 화면에 어떻게 보이는지 비교합니다.

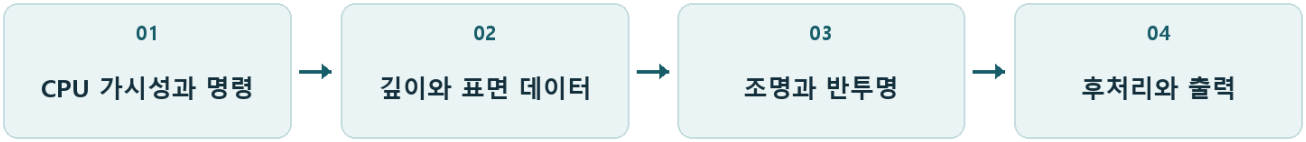
직교 투영은 평행한 방향을 유지합니다

직교 투영에서는 깊이가 커진다는 이유만으로 같은 물체가 작아지지 않습니다. 정면·측면 형상이나 배치를 읽을 때 유용하며, 원근 카메라와 같은 이미지가 나오는 설정은 아닙니다.

UV와 법선은 다른 정보입니다

UV는 표면의 어느 텍스처 위치를 읽을지, 법선은 표면이 어느 방향을 향하는지 나타냅니다.
꼭짓점의 위치를 화면에 옮기는 좌표 변환과 이 속성들을 계산에 전달하는 역할을 구분합니다.
일반적인 삼각형 처리와 Nanite의 세부 실행 차이는 CH17에서 설명합니다.

CPU의 준비가 화면 계산으로 이어지는 과정



개념도 / 실제 실행은 여러 갈래의 추가 작업을 포함합니다.

1 그릴 작업을 준비합니다

스레드는 프로그램이 명령을 이어 실행하는 흐름입니다. 게임 스레드는 주로 게임 상태를 갱신하고, 렌더 스레드는 CPU에서 장면을 그릴 명령을 준비합니다. RHI는 Render Hardware Interface의 약자로 엔진의 그래픽 명령을 DirectX·Vulkan 같은 그래픽 API에 연결하는 계층입니다. API는 다른 프로그램이나 장치의 기능을 사용하도록 정해 놓은 요청 방식입니다. 별도 RHI 스레드 사용 여부 등은 보충 주석 1을 참고합니다.

2 삼각형이 덮는 위치를 찾습니다

래스터화는 화면으로 옮긴 삼각형이 화면의 어느 위치를 덮는지 판정하는 과정입니다. 이때 검사하는 화면상의 위치를 샘플이라고 합니다. 한 픽셀 안에서 여러 위치를 검사할 수도 있습니다. 가려짐 등의 검사를 거쳐 남은 후보의 계산 결과가 최종 화면에 기여합니다.

3 꼭짓점 사이의 값을 이어 줍니다

보간은 알려진 값 사이에서 중간 위치의 값을 계산하는 것입니다. 삼각형 내부의 위치에서는 세 꼭짓점에 저장된 UV 등으로 그 위치에 필요한 값을 구합니다. 따라서 삼각형 한가운데에서도 텍스처의 어느 부분을 읽을지 정할 수 있습니다. 원근에 따른 보정 등 수학적 세부는 이 흐름을 이해한 뒤 읽어도 됩니다.

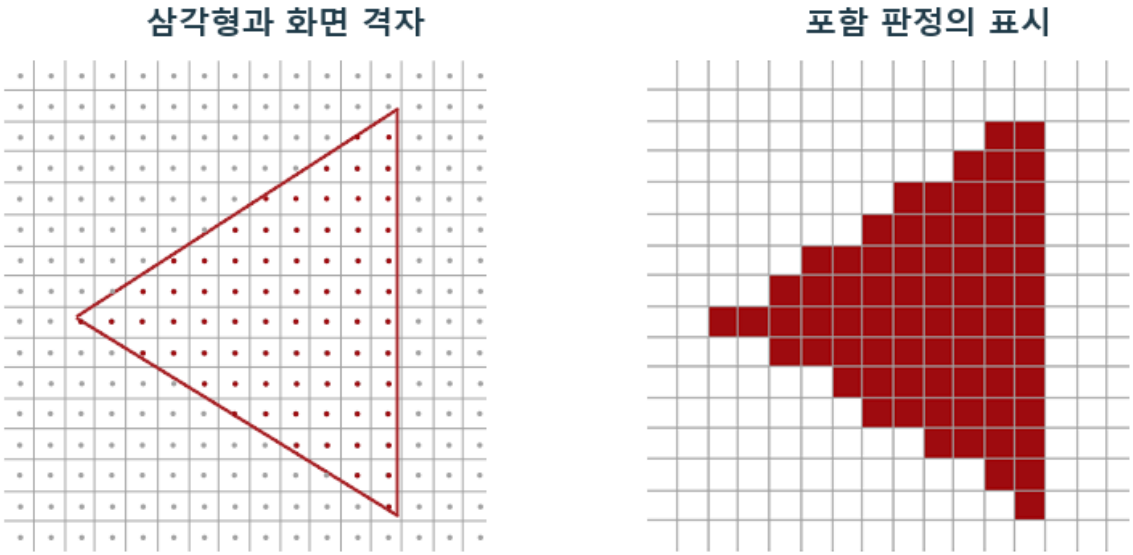
4 깊이와 표면 정보를 준비합니다

버퍼는 계산 중 사용할 데이터를 보관하는 메모리 공간입니다. 깊이 버퍼는 앞쪽 표면의 깊이를, GBuffer는 조명에 필요한 재질 정보를 담습니다. Depth Prepass는 깊이를 먼저 준비하는 단계이고 Base Pass는 주로 불투명 표면 정보를 준비하는 작업입니다. 이런 작업 단위를 렌더 패스라고 합니다. 자세한 데이터는 CH03·05에서 관찰합니다.

5 조명과 최종 화면을 만듭니다

표면의 방향과 반사 특성에 빛·그림자 등을 반영합니다. 유리처럼 뒤가 비치는 표면은 배경과 별도로 계산한 결과를 합성합니다. 이후 Bloom처럼 밝은 부분을 주변으로 퍼뜨리거나 화면 색을 조정하는 후처리도 적용합니다. 이 도식은 대표적인 의존 관계이며 실제 실행은 분기와 추가 작업을 포함합니다.

삼각형이 화면의 값으로 바뀌는 순간



외부 비교 그림 · 삼각형과 화면 격자, 그리고 포함 판정을 표시한 결과를 나란히 배치했습니다.

읽을 곳 왼쪽의 매끈한 삼각형 경계와 오른쪽의 격자에 맞춘 계단형 경계를 비교합니다. 이 그림은 래스터화와 앨리어싱의 관계를 설명하며 TAA 또는 TSR의 전후 결과가 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · CC BY 4.0 · 출처 [38]
원본 2점 나란히 배치 · 상단 표제 추가 · 색상 유지 · 이용 조건

먼저 화면을 덮는지 확인합니다

격자는 화면의 픽셀 영역입니다. 오른쪽의 빨간 칸은 삼각형의 포함 판정 결과를 보여 주는 예입니다. 실제 구현은 정해진 샘플 위치와 경계 규칙을 사용하므로 삼각형이 격자 칸과 조금 겹친다고 그 픽셀의 모든 샘플이 포함되는 것은 아닙니다.

그 위치에 쓸 값을 구합니다

삼각형 내부의 한 샘플에 대응하는 UV나 표면 방향은 꼭짓점 사이의 값을 이용해 계산합니다. 따라서 꼭짓점에만 정보가 있어도 내부의 텍스처 위치를 정할 수 있습니다. 실제 원근 투영에서는 화면에서 단순하게 평균하는 것과 다른 보정이 필요합니다.

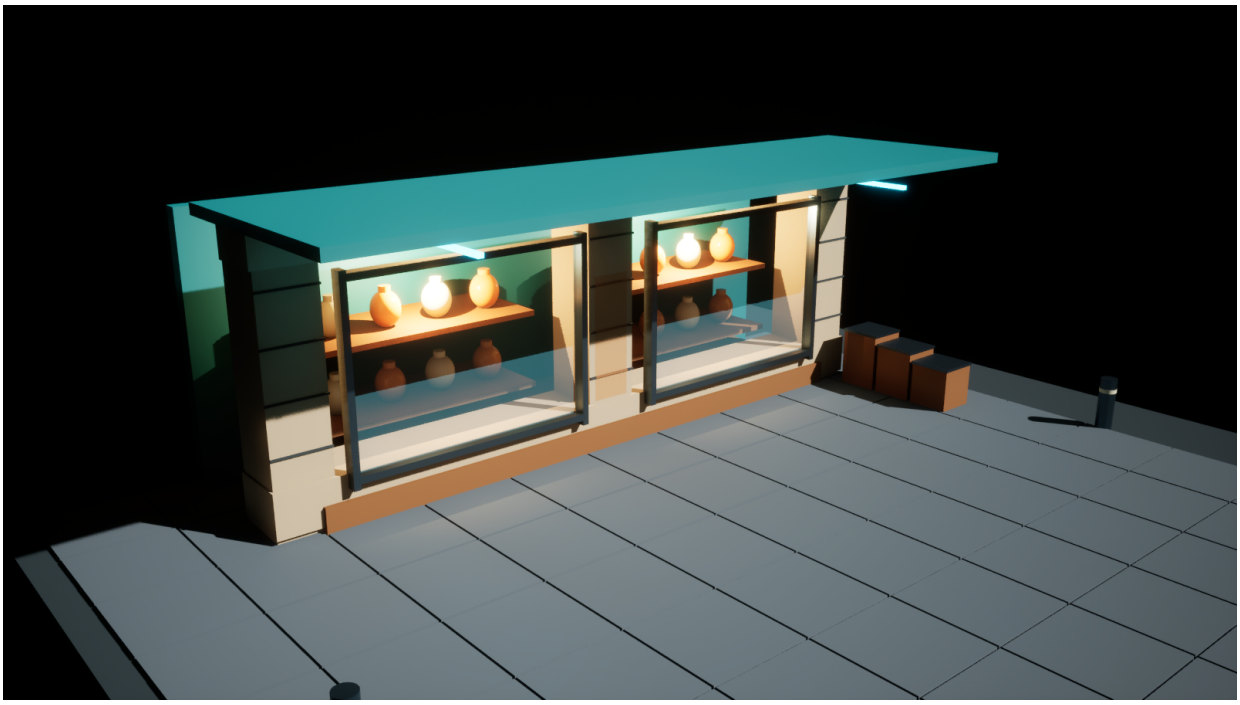
깊이 검사와 재질 계산으로 연결됩니다

포함된 샘플이 모두 최종 화면에 남는 것은 아닙니다. 앞쪽 표면에 가려지거나 다른 조건에 의해 제외될 수 있습니다. 살아남은 표면의 값과 조명·합성이 화면에 기여하며, 작업 순서와 조기 생략 조건은 경로에 따라 달라집니다.

아트 작업에서는 어디서 보일까

큰 삼각형에서도 텍스처가 연속적으로 보이는 이유를 보간으로 설명할 수 있습니다. 반대로 실루엣의 곡선은 정점의 수와 배치에 영향을 받습니다. 텍스처의 세밀함과 형상의 세밀함을 구분해 수정할 대상을 선택합니다.

씬에서 비교할 대상을 찾기



직접 엔진 캡처. 바닥·기둥, 두 쇼윈도와 내부 진열대, 처마 아래 발광 바를 관찰합니다.

1 벽과 바닥

01_OpaqueArchitecture 폴더에는 석재 기둥, 포장석과 금속 프레임이 있습니다. 같은 재질 색도 조명 방향과 표면 방향에 따라 Lit에서 다르게 보입니다. 포장석 틈은 실제 삼각형 형상이므로 삼각형 에지를 표시하는 Wireframe과 깊이에서도 확인할 수 있습니다.

2 쇼윈도 내부와 유리

02_OpaqueDisplay의 선반·도자기 앞에 03_Translucency의 GlassPane_1과 GlassPane_2가 놓여 있습니다. 유리 두 개만 숨기면 내부 물체와 조명 배치를 유지한 채 반투명 합성의 영향을 관찰할 수 있습니다.

3 표면 얼룩과 발광 바

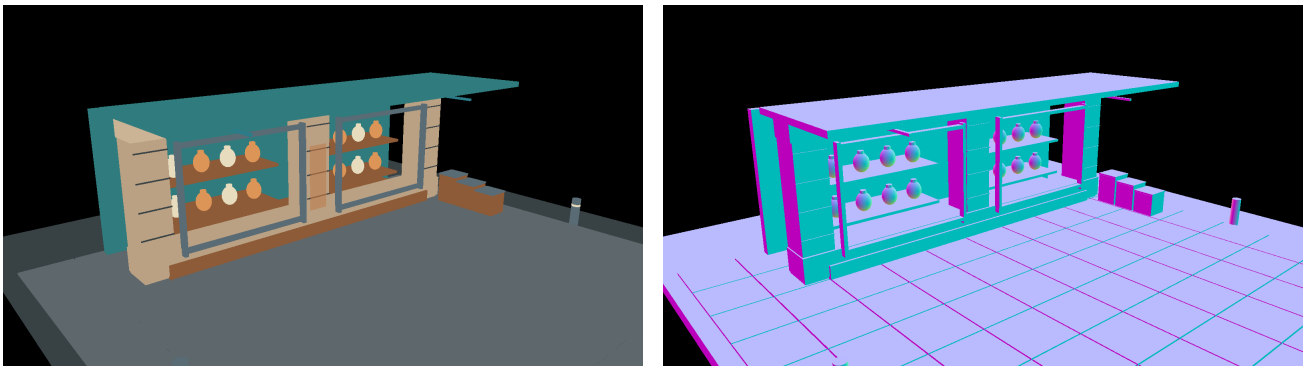
04_Decals의 WallPatina_DBuffer는 중앙 석재를 바꿉니다. 05_Signage의 발광 바는 표면 자체가 밝게 보이도록 하는 Emissive 입력을 사용합니다. 빛나는 표면, 주변을 비추는 광원, 화면에서 퍼지는 Bloom은 각각 영향을 분리해 확인할 수 있습니다.

에디터에서 시작하기

CH01 맵을 열고 장면의 배치 목록인 Outliner에서 CH01_ComparisonCamera를 선택합니다. Pilot은 뷰포트를 해당 카메라의 시점으로 보는 기능입니다. 06_Lighting과 08_PostProcess를 펼쳐 실습 대상을 확인합니다. 카메라와 수동 노출은 비교 동안 유지합니다. 에디터 구도와 실제 캡처 해상도는 별개이므로 비교 캡처는 모두 1600×900으로 맞춥니다.

Base Color와 법선을 분리해서 읽기

Base Color **World Normal**



직접 엔진 캡처 비교. 왼쪽 Base Color, 오른쪽 World Normal. 동일 장면의 다른 표면 데이터를 보여 줍니다.

Base Color에서 확인할 것

Viewport의 View Mode → Buffer Visualization → Base Color를 선택합니다. 석재와 도자기, 포장석의 입력 색이 어떻게 구분되는지 봅니다. Lit의 어두운 면이 이 화면에서도 어둡다면 재질 입력을, 이 화면에서는 비슷한데 Lit에서만 크게 다르면 조명·가림·반사 등을 다음 후보로 조사합니다.

World Normal에서 확인할 것

World Normal을 선택합니다. 표면 방향인 법선의 세 방향 성분을 색으로 표현하므로 방향이 다른 면은 색도 달라집니다. 구의 색이 연속적으로 변하는 것은 꼭짓점 사이의 표면 방향을 부드럽게 이어 계산하기 때문입니다. 벽을 회전하면 월드 법선이 달라집니다. 작은 표면 방향 변화를 이미지로 넣는 노멀 맵이 없어도, 면의 방향에 따라 서로 다른 색이 나타납니다.

직접 바꿔 보기

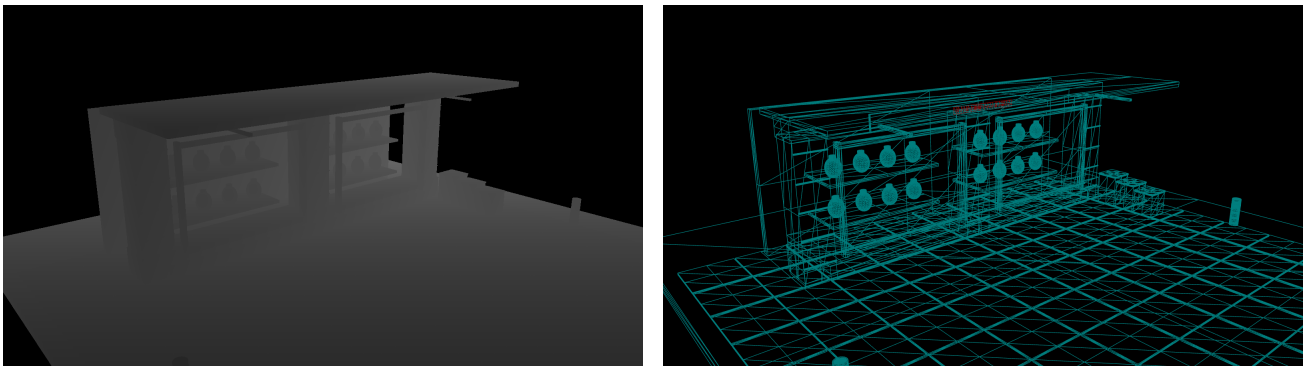
MI_CH01_Sandstone의 BaseColor만 바꾼 다음 두 버퍼를 다시 봅니다. 색 입력은 바뀌지만 메시와 법선을 바꾸지 않았으므로 World Normal은 같은 형태를 유지해야 합니다. 실습 뒤 원래 값으로 되돌립니다. 이 방식으로 바꾼 변수와 영향을 받은 데이터의 관계를 확인합니다.

버퍼 화면의 의미

이 이미지는 엔진의 진단 시각화입니다. GBuffer의 저장 데이터를 그대로 읽은 화면과는 표시 방식이 다를 수 있습니다. 특히 반투명 표면은 불투명 표면과 다른 경로로 처리되므로 Lit의 유리색이 모든 버퍼에 같은 방식으로 기록될 것이라 예상하지 않습니다.

깊이와 삼각형을 함께 관찰하기

Scene Depth **Wireframe**



직접 엔진 캡처 비교. 왼쪽 Scene Depth 진단 화면, 오른쪽 Wireframe. 색과 밝기는 표시용 표현입니다.

Scene Depth에서 확인할 것

Buffer Visualization → Scene Depth를 선택합니다. 가까운 볼라드, 앞쪽 포장석과 뒤쪽 진열대가 서로 다른 깊이로 표현되는지 확인합니다. 화면에 표시된 명암을 월드 거리 수치로 바로 읽지 않습니다. 하드웨어 깊이와 진단 화면의 변환 방식이 개입합니다.

유리와 깊이의 관계

일반적인 반투명 유리는 불투명 표면처럼 기본 깊이를 덮어쓰지 않습니다. 따라서 기본 Scene Depth에서 유리 뒤 물체의 깊이가 관찰될 수 있습니다. 선택한 물체의 깊이를 따로 기록하는 Custom Depth나 특정 반투명 기능은 별도 설정이므로 이 실습의 기본 깊이와 구분합니다.

Wireframe에서 확인할 것

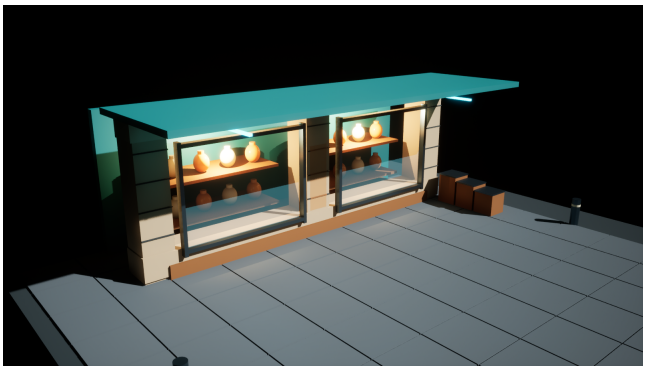
Alt+2 또는 View Mode → Wireframe을 선택해 삼각형 에지를 봅니다. 삼각형이 많아 보인다는 이유만으로 해당 장면의 병목을 정할 수는 없습니다. 표면이 화면을 덮는 크기, 재질 계산, 그림자와 가시성도 영향을 줍니다. Alt+4로 Lit에 돌아갑니다.

가려짐 실험

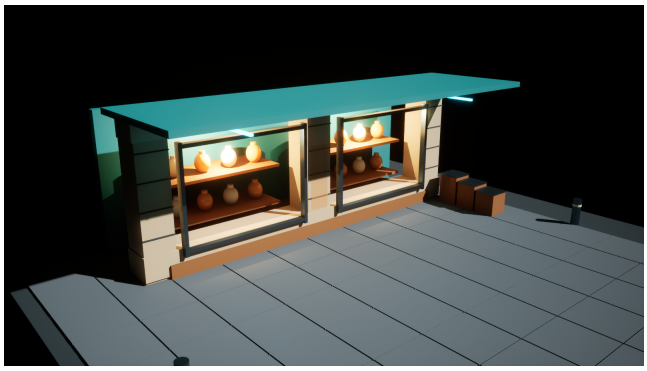
카메라 가까이에 불투명 큐브를 놓아 일부 도자기를 가립니다. Scene Depth가 앞쪽 큐브를 나타내는 위치와 Lit에서 가려지는 위치를 비교합니다. 깊이 검사로 어떤 표면 계산이 줄었는지까지 판단하려면 깊이를 먼저 준비한 작업과 가시성, GPU에 기록된 작업 구간을 추가로 확인합니다. Early Z의 조건은 CH05에서 설명합니다.

유리를 숨겨 반투명 합성을 확인하기

유리 표시



유리 숨김



직접 엔진 A/B 캡처. 왼쪽 유리 표시, 오른쪽 GlassPane_1·2만 숨김. 카메라·조명·노출은 동일합니다.

실습 절차

Lit에서 원본 화면을 확인합니다. Outliner의 03_Translucency 폴더를 펼쳐 GlassPane_1과 GlassPane_2를 선택하고 에디터 가시성만 끕니다. 선반, 도자기, 프레임과 광원은 그대로 둡니다. 같은 카메라로 다시 캡처한 뒤 유리 표시 상태를 복원합니다.

관찰 결과를 설명하기

두 이미지에서 쇼윈도 내부의 색과 대비, 유리 표면에 나타나는 반사 기여를 비교합니다. 이 실습의 유리는 Opacity 0.28, Roughness 0.10과 청록색 Base Color를 가진 뒤의 색을 합성하는 단순 Translucent 재질입니다. 물리적 두께에 따른 흡수나 굴절을 정교하게 검증하는 실험은 이후 Translucency 챕터에서 다룹니다.

어느 단계를 조사할까

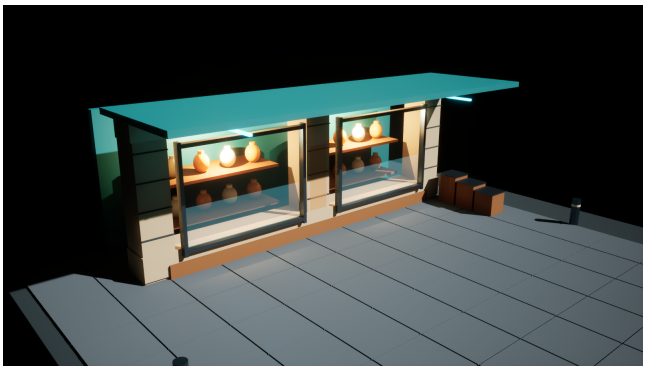
유리만 숨겼을 때 문제가 사라지면 반투명 셰이딩·정렬·합성 및 관련 반사를 조사할 근거가 됩니다. 불투명 배경이 동일하게 그려졌는지도 함께 확인합니다. Lumen처럼 다른 표면의 가시성과 화면 추적에 반응하는 기능 때문에 변화가 반투명 패스 하나에만 국한되지는 않을 수 있습니다.

성능 검증으로 이어가기

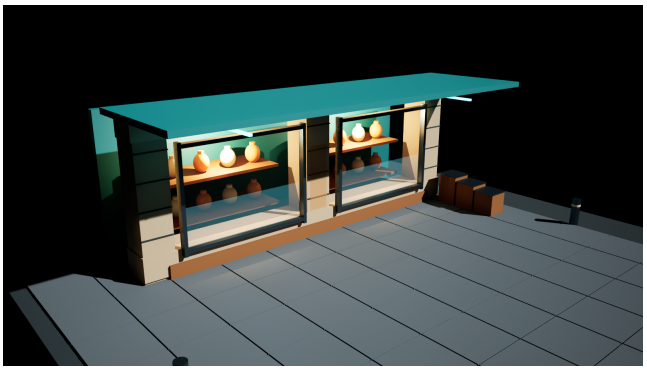
stat gpu 또는 GPU 작업을 시간축에 펼친 분석 화면에서 Translucency 계열과 전체 GPU 시간을 기록합니다. A와 B의 측정 순서를 바꿔 반복합니다. 셰이더 준비와 로딩을 마쳐 안정화하는 워밍업 구간은 별도로 기록합니다. 이 문서의 비교 이미지는 시각적 변화의 증거이며, 고해상도 캡처 중의 프레임 시간을 성능 수치로 사용하지 않습니다.

Bloom과 발광을 구분하기

Bloom 1.0



Bloom 0.0



직접 엔진 A/B 캡처. 왼쪽 Bloom Intensity 1.0, 오른쪽 0.0. 발광 재질과 실제 광원은 유지했습니다.

실습 절차

08_PostProcess의 CH01_FixedExposure를 선택합니다. Bloom Intensity 옆의 Override 체크를 유지한 채 값을 조정합니다. Override는 이 볼륨의 설정을 실제로 적용한다는 뜻입니다. 값을 1.0에서 0.0으로 바꿉니다. 카메라 노출, M_CH01_SignEmission과 진열장 Point Light는 수정하지 않습니다. 캡처 후 값을 1.0으로 되돌립니다.

관찰할 차이

처마 아래 밝은 바의 주변으로 퍼지는 화면 효과를 비교합니다. Bloom을 줄여도 발광 재질의 밝은 중심 자체는 남습니다. 그린 화면을 가공하는 후처리의 번짐과 장면 조명의 기여를 따로 생각하면, 간판이 과하게 번질 때 어떤 값을 먼저 바꿀지 판단하기 쉽습니다.

발광이 주변에 미치는 영향

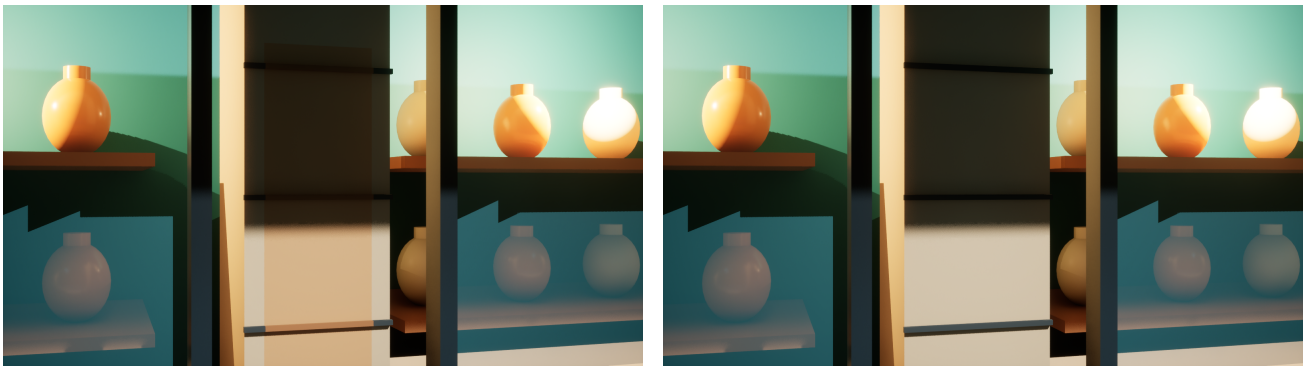
이 씬은 Lumen을 사용하므로 Emissive가 간접광에 기여할 수 있습니다. 진열장에는 별도의 WarmDisplayLight도 있습니다. Bloom만 바꾸는 실험에서 두 광원의 에너지를 동시에 바꾸지 않아야 비교 의도가 유지됩니다.

아트 실무의 진단

밝은 간판을 고치려다 Base Color를 어둡게 만들기 전에 Bloom, 수동 노출과 Emissive 값을 각각 확인합니다. 톤매핑은 계산한 장면의 밝기를 디스플레이에 보여 줄 범위로 바꾸는 처리입니다. 이때 높은 밝기의 차이가 압축되어 보이는 영향도 함께 판단합니다. 최종 출력 색은 Base Color 값만으로 결정되지 않습니다.

데칼과 조명에서 생긴 변화를 찾기

데칼 표시 근접 **데칼 숨김 근접**



직접 엔진 A/B 근접 캡처. 같은 중앙 기둥에서 데칼 표시와 숨김을 비교합니다.

데칼 실험

04_Decals의 WallPatina_DBuffer만 숨기고 원본 Lit와 중앙 기둥을 비교합니다. 이 데칼은 색과 Roughness 계열 표면 값을 수정합니다. DBuffer 경로에서는 Base Pass 전에 데칼 정보를 준비하고 수신 머티리얼이 이를 적용합니다. 데칼이 최종 색을 화면 위에 단순히 붙이는 것처럼만 이해하면 반사 변화의 원인을 놓치기 쉽습니다.

조명 실험

데칼을 다시 표시하고 View Mode → Lighting Only를 선택합니다. 표면의 색을 중립 재질로 바꾸어 빛과 음영 구조를 관찰합니다. 이 화면은 최종 Lit의 밝기를 정확히 분해한 수치가 아니라 조명 상태를 읽기 위한 진단 표현입니다.

같은 위치를 여러 데이터로 확인하기

중앙 기둥이 어둡다면 Base Color, World Normal, Lighting Only, 데칼 가시성을 차례로 확인합니다. 입력 색, 방향, 빛과 표면 수정 중 어떤 변화와 연결되는지 기록합니다. 서로 다른 진단 모드의 픽셀 밝기를 그대로 빼서 조명 에너지라고 해석하지 않습니다.

실무 예시

모듈 벽면의 얼룩은 데칼로 변주하면서 기본 재질은 공유할 수 있습니다. 투영 박스를 좁히고 겹침을 줄였을 때 시각적 범위가 유지되는지 확인하고, 비용은 별도의 전후 프로파일링으로 확인합니다.

제작 과정을 직접 재현하기

1 기본 씬 열기

RenderingLab.uproject를 UE 5.8.4에서 엽니다. CH01 레벨과 CH01_ComparisonCamera를 선택합니다. CH01은 Deferred, Lumen, VSM, TSR과 Substrate Blendable 시작 설정을 사용합니다. 이 비교를 위해 렌더러 설정을 바꿀 필요는 없습니다.

2 불투명 재질 만들기

CH01의 Materials 폴더에서 MI_CH01_Sandstone과 MI_CH01_MetalFrame을 엽니다. 부모 재질 M_LabSurface는 색·금속성·거칠기를 설정값으로 노출한 기본 표면입니다. Material Instance는 이 구조를 공유하며 값을 바꾸는 변형입니다. 금속 프레임은 Metallic 0.85, Roughness 0.26이고 석재는 Metallic 0, Roughness 0.82입니다. 학습용 수치이며 측정된 실물 재질 데이터는 아닙니다.

3 유리와 발광 구성하기

M_CH01_Glass는 뒤의 색과 합성하는 Translucent 재질입니다. 표면의 조명을 계산할 방식으로 Surface ForwardShading을 선택했습니다. Base Color (0.10, 0.32, 0.38), Opacity 0.28, Roughness 0.10을 연결합니다. M_CH01_SignEmission의 Emissive 입력은 (0.04, 3.5, 5.0)입니다. 이 숫자는 재질에 넣은 설정값이며 저장된 스크린샷의 픽셀 숫자와 같지 않습니다. 최종 색에는 빛과 화면 표시용 변환이 추가됩니다.

4 빛과 비교 조건 맞추기

Sun_Key와 WarmDisplayLight_1·2가 직접광을 만듭니다. CH01_FixedExposure는 Manual 노출과 물리 카메라 노출 비적용을 사용합니다. 비교 중 라이트와 노출을 고정합니다. 캡처 도구는 내부 렌더링 크기를 정하는 Screen Percentage를 100으로 고정합니다. 부하에 따라 크기를 자동 변경하는 Dynamic Resolution은 끄고, 1600×900으로 출력합니다.

5 캡처 재현하기

File → Execute Python Script에서 Content/Python/capture_ch01.py를 실행하면 이 챕터의 진단·A/B 이미지를 순서대로 저장합니다. 이 도구는 전용 에디터 세션에서 실행하고, 실행 전 다른 작업을 저장합니다. 완료 시 에디터가 종료됩니다. 원본과 같은 파일 이름으로 캡처를 갱신합니다.

생성 도구의 범위

build_ch01.py는 CH01 레벨의 액터를 재구성하는 제작 도구입니다. 기존 CH01을 수정한 뒤 다시 실행하려면 변경을 커밋하거나 복사본을 확보합니다. 일반적인 학습과 재캡처에는 이미 저장된 레벨과 capture_ch01.py를 사용합니다.

검증 기록과 다음 실습

이 챕터에서 확인한 것

UE 5.8.4와 NVIDIA GeForce RTX 3070 Ti의 DX12 에디터에서 CH01을 렌더링했습니다. Lit, Base Color, World Normal, Scene Depth, Roughness, Wireframe, Lighting Only와 유리·Bloom·데칼 A/B 및 근접 화면을 촬영했습니다. 원본 이미지 12개와 CaptureManifest.json은 Docs/Images/CH01에 있습니다.

관찰과 측정을 구분하기

본 실습은 각 기능이 화면에 미치는 영향을 확인합니다. 처리 비용을 결론내리려면 대상 기기에서 워밍업한 동일 카메라 경로를 반복 실행해야 합니다. 고해상도 캡처는 렌더링을 동기화하고 작업량도 바꿀 수 있으므로 이 챕터에 임의의 ms 개선값을 붙이지 않았습니다.

검증 체크

① Base Color만 바꿨을 때 World Normal의 형태가 유지되는가. ② 유리만 숨겼을 때 내부 선반과 도자기는 남는가. ③ Bloom만 껐을 때 발광 바의 중심은 남는가. ④ 데칼을 숨겼을 때 중앙 기둥의 표면 변화가 사라지는가. ⑤ 비교마다 카메라·해상도·노출을 유지했는가.

문제가 생기면

장면이 검게 보이면 셰이더·거리장 준비가 끝났는지, Sun_Key와 노출이 유효한지 확인합니다. 유리가 불투명하면 올바른 머티리얼과 Blend Mode를 봅니다. 진단 모드 뒤에는 Alt+4로 Lit에 돌아갑니다. 캡처 파일이 누락되면 Saved/Logs/CH01Capture.log와 CaptureManifest.json의 complete 값을 확인합니다.

핵심 정리

메시 배치, 표면 입력, 조명, 반투명과 후처리는 서로 다른 데이터를 바꾸면서 최종 화면에 연결됩니다. 증상이 보이는 위치를 여러 진단 모드와 한 변수의 전후 화면으로 확인하면 조사할 패스와 설정을 좁힐 수 있습니다. 다음 CH02에서는 표면과 조명을 평가하는 시점이 다른 Forward와 Deferred를 비교합니다.

Epic 공식 자료

Viewport Modes

Decal Materials

Lit Translucency

Render Dependency Graph

Scripting the Editor Using Python

Parallel Rendering Overview

Direct3D Rasterizer Stage

02 Forward와 Deferred

챕터 학습 목표

표면과 조명을 평가하는 방식의 차이가 기능 지원과 비용에 어떻게 연결되는지 이해합니다. VR, 모바일, 데스크톱 등 목표 환경에서 필요한 화질·AA·조명 기능을 기준으로 렌더러 선택과 에셋 제작 조건을 판단합니다.

예제 학습 목표

동일한 장면과 공통 지원 설정에서 광원 겹침을 바꿔 Forward와 Deferred의 화면 및 패스 비용을 비교합니다.

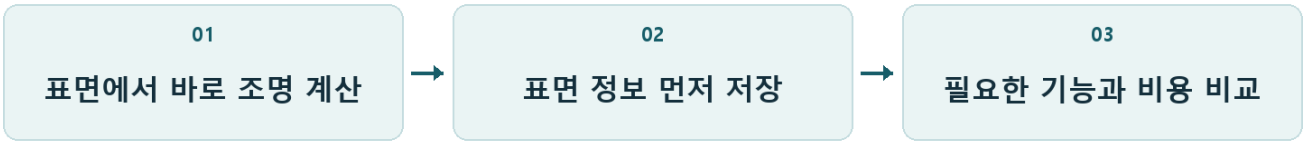


그림 02 / Forward와 Deferred 개념도

원리

Forward와 Deferred는 표면과 빛을 계산하는 순서가 다른 렌더링 방식입니다. Forward는 물체의 표면을 그리면서 조명을 계산합니다. Deferred는 표면 정보를 먼저 저장하고, 뒤의 조명 단계가 그 정보를 읽습니다. 이 문서에서는 PC·콘솔에서 사용하는 데스크톱 경로를 설명합니다.

작동 과정

한 벽을 여러 라이트가 비추더라도 모든 라이트를 항상 전부 계산할 필요는 없습니다. 엔진은 화면과 공간을 나누어 해당 위치에 영향을 주는 광원을 추립니다. UE의 Forward도 이런 선별을 사용합니다. 반사 환경을 미리 담아 두는 Reflection Capture 역시 영향을 주는 범위에 따라 선택됩니다.

아트 실무

VR에서는 머리 움직임에 따른 가장자리의 안정성이 중요합니다. 계단처럼 보이는 가장자리를 줄이는 처리를 안티앨리어싱 또는 AA라고 합니다. MSAA는 한 픽셀 안의 여러 위치에서 도형의 경계를 검사하는 방식입니다. 시간적 누적 방식인 TAA·TSR은 CH23·24에서 설명합니다. 렌더러를 정할 때 필요한 AA·조명·반사 기능이 함께 지원되는지 확인합니다.

실무에서 비교하고 검증하기

성능 비용

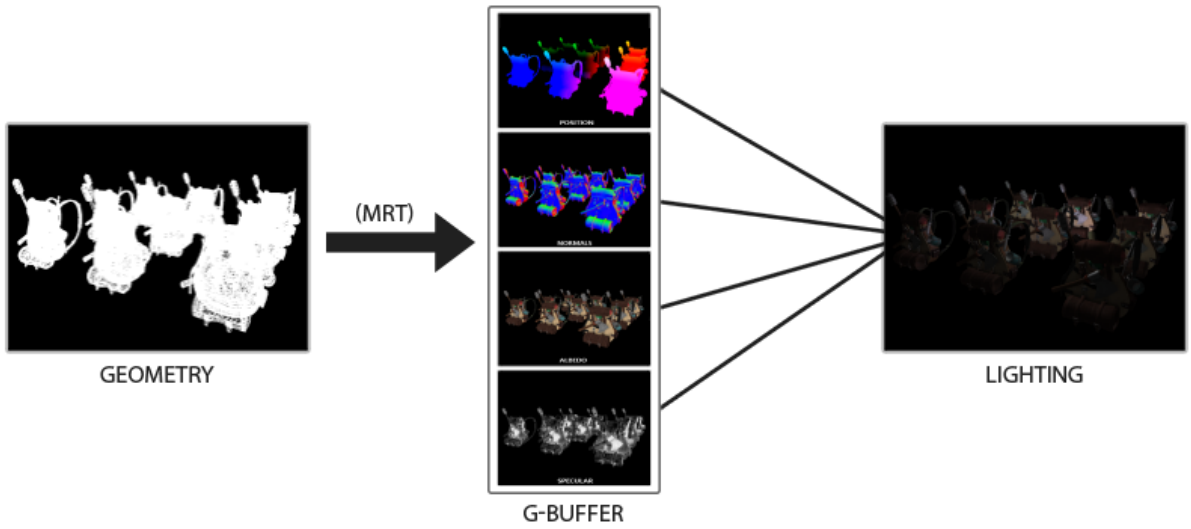
Deferred는 화면 크기의 표면 데이터를 메모리에 쓰고 다시 읽습니다. Forward는 표면을 그릴 때 필요한 광원을 함께 처리합니다. 무엇이 빠른지는 광원 겹침, 그림자, 재질과 지원 기능에 따라 달라집니다. 특정 장면의 결과를 다른 프로젝트의 성능 보장으로 사용하지 않습니다.

언리얼 실습

프로젝트 복사본에서 Project Settings → Rendering → Forward Shading을 바꾸고 재시작합니다. 두 방식이 공통 지원하는 조명·반사·AA를 맞춥니다. 같은 구도에서 겹치는 광원 수만 바꾸고 stat gpu의 Base Pass와 조명 관련 시간을 기록합니다. Base Pass는 주로 불투명 표면 정보를 준비하는 작업이며 CH03에서 이어 설명합니다.

핵심 요약

계산 순서의 차이를 필요한 기능과 연결해 렌더러를 선택합니다. 비교할 때는 한쪽에서 빠진 기능 때문에 빨라진 것인지도 확인합니다.



외부 참고 그림 · Geometry Pass에서 얻은 여러 표면 버퍼를 Lighting Pass에서 사용합니다.

읽을 곳 가운데 버퍼의 위치·법선·색 정보가 오른쪽의 조명 결과에 어떻게 모이는지 봅니다. OpenGL 학습용 구현이므로 UE의 실제 GBuffer 배치와 동일하다고 해석하지 않습니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · CC BY 4.0 · 출처 [39]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

공식 자료와 이어 읽기

[01] Rendering Settings
[04] Forward Shading Renderer
CH01 CH23 CH24 용어집 보충 주석

03 GBuffer와 표면 정보

챕터 학습 목표

최종 색을 만드는 재질 데이터와 조명 결과를 구분합니다. 색·법선·거칠기 문제를 진단하고, 해상도와 저장 형식이 메모리·대역폭에 미치는 영향을 이해해 재질과 렌더링 설정을 함께 판단합니다.

예제 학습 목표

같은 벽의 거칠기 입력을 바꾸고 버퍼 시각화와 내부 해상도 비교로 입력 데이터와 비용의 관계를 확인합니다.

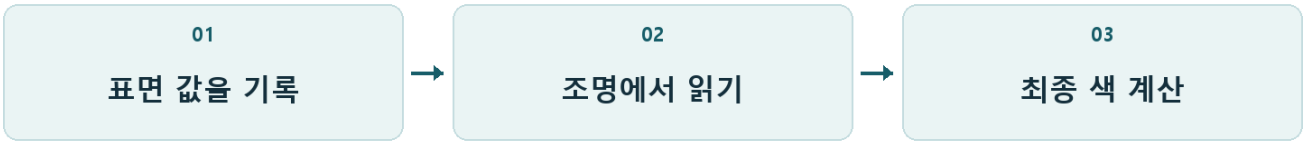


그림 03 / GBuffer와 표면 정보 개념도

원리

버퍼는 계산 중 필요한 데이터를 보관하는 메모리 공간입니다. GBuffer는 Deferred 조명이 읽을 표면 정보의 묶음입니다. 재질의 기본 색인 Base Color, 반사가 퍼지는 정도인 Roughness, 표면이 향하는 방향인 법선 등이 여기에 관여합니다. 최종 화면 색과는 역할이 다릅니다.

작동 과정

Base Pass는 화면에 보이는 불투명 표면의 정보를 기록하는 대표 작업입니다. 조명 단계는 그 값을 읽어 표면에 빛이 어떻게 반사될지 계산합니다. 깊이를 저장한 별도 데이터인 Scene Depth와 카메라 정보를 이용해 표면 위치를 알아낼 수도 있습니다. 어떤 값을 몇 개의 채널에 담는지는 플랫폼과 저장 형식에 따라 달라집니다.

아트 실무

벽이 유난히 번들거리면 Roughness 화면을 먼저 봅니다. 밝은 반사 부분의 방향이 이상하면 World Normal을 봅니다. 노멀 맵은 작은 표면 방향 변화를 텍스처로 표현하는 방법입니다. 노멀 맵에 쓰는 방향 기준이 메시와 맞지 않으면 반사가 어색해질 수 있습니다. 방향 기준의 세부 구현은 용어집의 탄젠트를 참고합니다.

실무에서 비교하고 검증하기

성능 비용

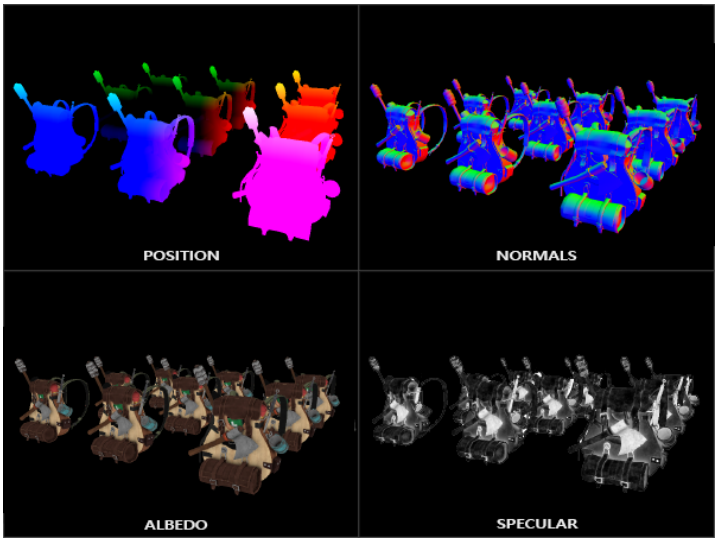
메모리 사용량은 저장할 픽셀 수와 픽셀당 데이터 크기에 영향을 받습니다. 가정한 저장량이 32바이트/픽셀이면 1080p는 약 63.3 MiB이며 4K에서는 네 배입니다. 실제 UE 버퍼 크기를 뜻하는 수치는 아닙니다. 대역폭은 일정 시간에 데이터를 옮길 수 있는 양이며, 같은 데이터를 자주 읽고 쓰면 이 한도도 중요해집니다.

언리얼 실습

같은 벽 재질의 설정값 변형인 Material Instance를 두 개 만들고 Roughness를 0.2와 0.8로 둡니다. View Mode → Buffer Visualization에서 Roughness와 Base Color를 비교합니다. 다음에는 내부 렌더링 해상도만 바꿔 Base Pass와 전체 GPU 시간을 확인합니다. 머티리얼 인스턴스의 제작 방식은 CH07에서 설명합니다.

핵심 요약

재질 입력이 잘못됐는지와 조명 이후에 문제가 생겼는지를 분리합니다. 노드가 늘어난다고 저장 바이트까지 항상 늘어나는 것은 아닙니다.



외부 시각화 이미지 · 같은 장면의 위치·법선·Albedo·Specular를 분리해 표시한 GBuffer 예시입니다.

읽을 곳 왼쪽 위는 위치에 따라, 오른쪽 위는 표면 방향에 따라 색이 달라집니다. 아래쪽은 재질의 색과 반사 입력입니다. 진단용 색을 최종 외관이나 UE 채널 구성으로 읽지 않습니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · CC BY 4.0 · 출처 [39]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

공식 자료와 이어 읽기

[02] Render Dependency Graph
[05] Programming with Substrate GBuffer Formats
CH07 CH11 용어집 보충 주석

04 DBuffer와 데칼

챕터 학습 목표

데칼이 수신 표면을 바꾸는 원리와 적용 시점을 이해합니다. 벽의 오염·손상·표식으로 반복을 줄일 때 투영 범위, 수신 설정, 겹침이 표현과 비용에 미치는 영향을 판단합니다.

예제 학습 목표

벽 데칼의 개수와 투영 박스 범위를 바꿔 표면 변화, 잘못 묻는 영역과 관련 패스의 비용을 확인합니다.

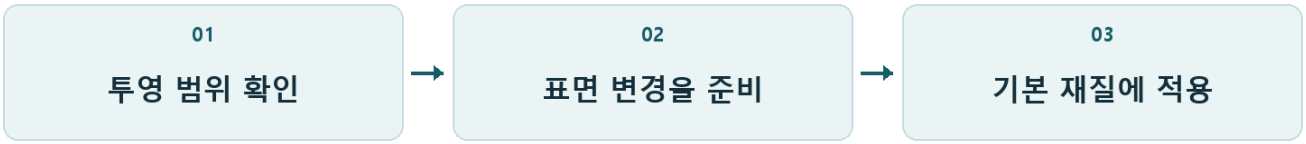


그림 04 / DBuffer와 데칼 개념도

원리

데칼은 벽의 얼룩이나 표식을 다른 표면 위에 투영하는 표현입니다. DBuffer는 데칼이 바꿀 색·거칠기·표면 방향 등의 값을 잠시 모아 두는 데이터입니다. 이 방식에서는 벽의 기본 재질을 계산하는 Base Pass가 데칼 정보를 읽어 표면에 반영합니다.

작동 과정

데칼 Actor의 박스는 투영할 공간 범위를 정합니다. 엔진은 깊이 정보로 그 안의 표면 위치를 찾고 데칼의 어느 부분을 적용할지 계산합니다. 별도 방식인 GBuffer 데칼은 Base Pass 뒤, 조명 전에 기존 표면 데이터에 합성합니다. 두 방식의 적용 시점과 지원 조건이 같다고 가정하지 않습니다.

아트 실무

모듈 벽 여러 개가 같은 재질을 쓰더라도 데칼로 빗물 자국과 손상을 다르게 줄 수 있습니다. 뒤의 소품에 얼룩이 함께 묻으면 투영 박스 깊이를 줄입니다. Receives Decals는 물체가 데칼을 받을지, Decal Response는 재질이 어떤 변경을 받을지 정하는 설정입니다.

실무에서 비교하고 검증하기

성능 비용

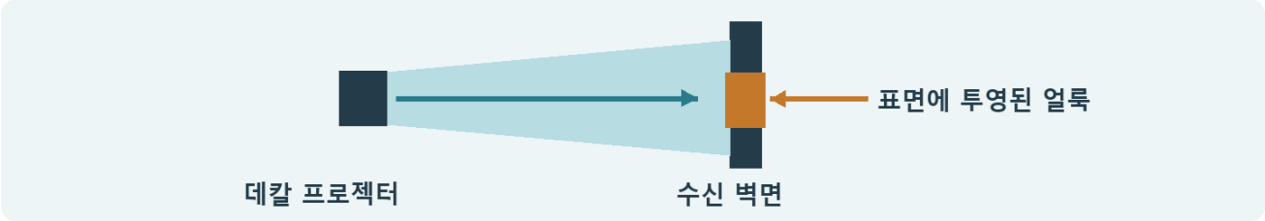
화면을 덮는 면적과 겹치는 데칼 수가 늘면 같은 위치에서 처리할 작업도 늘어납니다. DBuffer가 필요로 하는 깊이 준비 작업의 비용도 확인합니다. 이미 다른 기능 때문에 깊이를 먼저 준비하고 있다면, 그 비용 전체를 데칼 하나의 비용으로 계산하지 않습니다.

언리얼 실습

Material Domain을 Deferred Decal로 정한 얼룩 재질을 벽에 투영합니다. Domain은 그 재질을 어떤 용도로 사용할지 정하는 구분입니다. 한 개와 겹친 여덟 개를 비교한 뒤, 보이는 얼룩을 유지하면서 투영 박스만 줄입니다. 데칼 관련 패스와 Base Pass의 시간을 함께 기록합니다.

핵심 요약

데칼의 수뿐 아니라 투영 범위와 수신 표면을 관리합니다. DBuffer 값을 재질에서 직접 조합한다면 기본 적용과 중복되지 않는지도 확인합니다.



직접 제작 설명도 / 데칼 볼륨과 수신 표면이 만나는 위치에 표면 수정값을 적용합니다. 데칼의 면적·겹침과 수신 설정을 함께 확인합니다.

공식 자료와 이어 읽기

[05] Programming with Substrate GBuffer Formats

[06] Decal Materials

CH03 CH06 용어집 보충 주석

05 Depth와 Early Z

챕터 학습 목표

가려진 표면을 어떤 시점에 제외할 수 있는지 이해합니다. 밀집된 건축물·실내·식생에서 깊이 프리패스와 Early Z가 줄이는 작업을 구분하고, 가림 효과와 선행 비용을 함께 판단합니다.

예제 학습 목표

불투명 가림판으로 패널의 노출 범위를 바꾸며 Scene Depth와 PrePass·Base Pass의 변화를 확인합니다.

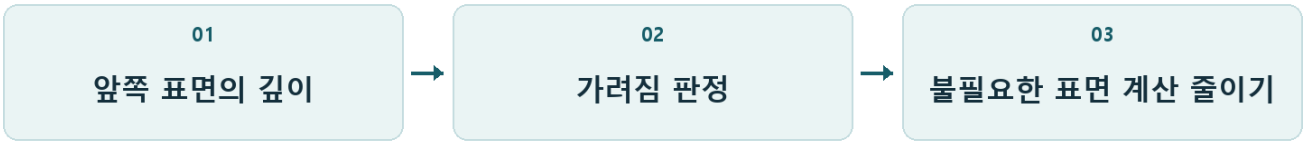


그림 05 / Depth와 Early Z 개념도

원리

깊이 버퍼는 화면 위치마다 앞쪽 표면의 깊이를 저장합니다. 새 삼각형이 이미 그린 표면 뒤에 있다면 최종 화면에 기여하지 않을 수 있습니다. Early Z는 가능한 경우 이런 가려짐을 먼저 검사해 비싼 표면 계산을 줄이는 GPU 동작입니다. Z는 깊이 축을 가리키는 관용적 이름입니다.

작동 과정

Depth Prepass는 주요 표면의 깊이를 먼저 준비하는 단계입니다. 뒤의 작업은 그 값을 읽어 가려진 후보를 제외할 수 있습니다. 삼각형이 덮는 화면 위치에서 만들어지는 처리 후보를 프래그먼트라고 부릅니다. 같은 화면 픽셀에도 여러 삼각형의 프래그먼트가 도착할 수 있으며, 모두 최종 색으로 남는 것은 아닙니다.

아트 실무

벽 뒤 장식은 벽의 깊이로 가려질 수 있습니다. 반면 잎 카드처럼 구멍이 있는 표면은 먼저 구멍 여부를 알아야 합니다. Masked 재질은 텍스처 등의 값으로 해당 위치를 남길지 버릴지 정합니다. 구멍이 많은 카드와 꽉 찬 벽의 깊이 처리 효과가 다른 이유입니다.

실무에서 비교하고 검증하기

성능 비용

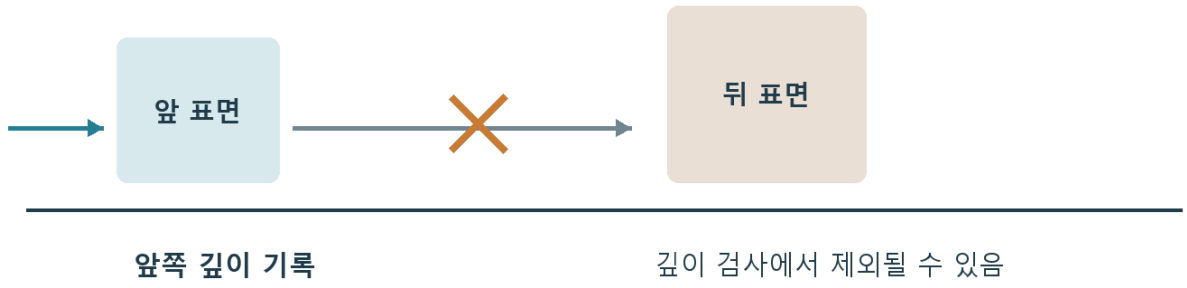
깊이를 먼저 준비하는 일에도 메시 처리와 메모리 쓰기가 필요합니다. 뒤에서 줄이는 표면 계산이 크면 이득이 날 수 있지만, 단순한 장면에서는 선행 작업의 부담도 봐야 합니다. 깊이 화면의 밝기는 진단용 표시이므로 그 숫자를 카메라와 물체 사이의 실제 거리로 바로 읽지 않습니다.

언리얼 실습

불투명 가림판 뒤에 복잡한 재질의 패널을 둡니다. 패널 일부가 계속 보이도록 하여 오브젝트 전체가 제외되는 영향을 줄입니다. 가리는 면적만 바꾸며 Scene Depth와 PrePass·Base Pass 시간을 기록합니다. 오브젝트 전체를 제외하는 컬링은 CH18에서 구분해 설명합니다.

핵심 요약

깊이 준비의 추가 작업과 줄어든 표면 계산을 함께 측정합니다. 화면에 안 보이는 물체라도 모든 비용이 자동으로 사라지는 것은 아닙니다.



직접 제작 설명도 / 앞쪽 불투명 표면의 깊이를 이용해 뒤쪽 후보를 거릅니다. 실제 셰이더 실행 생략 여부와 깊이 검사 시점은 재질·경로에 따라 달라집니다.

공식 자료와 이어 읽기

[02] Render Dependency Graph
[03] Introduction to Performance Profiling and Configuration
[06] Decal Materials
[30] Visibility and Occlusion Culling
CH06 CH18 용어집 보충 주석

06 Overdraw와 화면 겹침

챕터 학습 목표

화면의 같은 위치를 반복 처리하는 비용을 이해합니다. 식생, 파티클, 머리카락과 겹친 투명 표면에서 화면 면적·겹침·재질 계산량을 기준으로 형태와 표현 방식을 조정합니다.

예제 학습 목표

큰 잎 카드와 윤곽을 다듬은 카드를 비교해 삼각형 수, 커버리지와 GPU 비용이 함께 달라지는지 확인합니다.

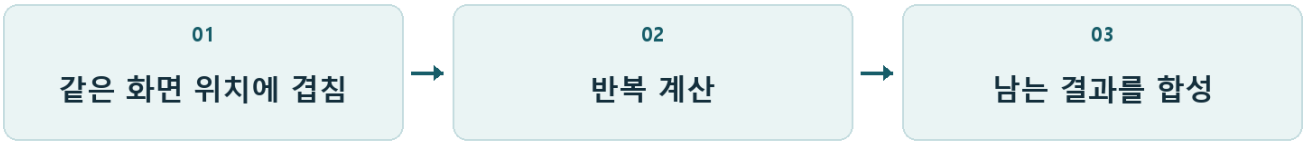


그림 06 / Overdraw와 화면 겹침 개념도

원리

Overdraw는 같은 화면 위치에서 여러 표면 후보를 반복 처리하는 현상입니다. 잎 카드 열 장이 겹치면 한 위치에 여러 삼각형의 계산 후보가 생깁니다. 어느 후보를 일찍 버릴 수 있는지는 불투명·구멍 마스크·반투명 재질의 처리 방식에 따라 달라집니다.

작동 과정

Opaque는 뒤를 가리는 불투명 표면입니다. Masked는 위치별로 남길지 버릴지 정하고, Translucent는 뒤의 색이 비치는 정도를 계산해 합성합니다. 불투명 앞면은 깊이로 뒤를 가릴 수 있습니다. 반투명 층끼리는 여러 층의 색이 필요해 처리가 반복되기 쉽습니다.

아트 실무

큰 사각형 한가운데 작은 잎만 그리면 투명한 주변 영역에도 마스크를 확인하는 작업이 필요할 수 있습니다. 잎 윤곽에 맞게 메시를 다듬으면 삼각형은 늘어도 불필요한 화면 영역은 줄일 수 있습니다. 근거리의 윤곽, 원거리의 깜빡임과 그림자까지 함께 비교합니다.

실무에서 비교하고 검증하기

성능 비용

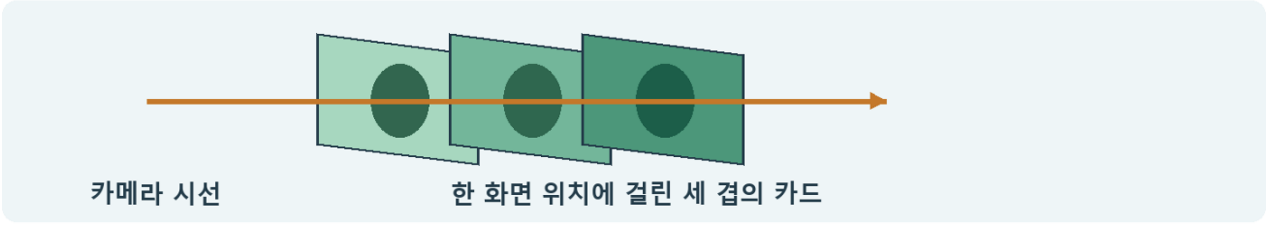
겹친 면적, 층 수, 한 번 처리할 때의 재질 계산량이 함께 중요합니다. Shader Complexity는 셰이더 명령 수 등을 바탕으로 한 진단 표현이며 실행 시간 자체가 아닙니다. Quad Overdraw는 GPU가 작은 화면 묶음으로 처리할 때 생기는 비효율을 살피는 모드입니다. 필요할 때 켜 보고 실제 시간과 대조합니다.

언리얼 실습

같은 잎 이미지로 큰 카드와 윤곽을 다듬은 카드를 만듭니다. 배치와 조명을 맞춘 뒤 Shader Complexity와 지원되는 Quad Overdraw를 확인합니다. 근거리·원거리에서 GPU 시간과 외관을 기록합니다. Nanite 메시는 전용 Overdraw 시각화도 사용합니다.

핵심 요약

삼각형 수 하나로 식생의 효율을 판단하지 않습니다. 실제로 반복 처리하는 화면 영역을 함께 봅니다.



직접 제작 설명도 / 동일한 화면 위치에 카드가 여러 겹 놓입니다. 보이지 않는 층까지 처리하면 비용이 늘 수 있으며, 실제 비용은 재질과 깊이 검사 조건에 달려 있습니다.

공식 자료와 이어 읽기

[02] Render Dependency Graph
[07] Viewport Modes
[19] Nanite Virtualized Geometry
CH05 CH12 용어집 보충 주석

07 PBR과 재질 입력

챕터 학습 목표

재질의 고유 특성과 장면 조명의 역할을 구분해 여러 환경에서도 일관된 재질을 만듭니다. 금속·비금속·오염·도장면의 입력을 설계하고, 어색한 외관이 텍스처와 조명 중 어디에서 생겼는지 판단합니다.

예제 학습 목표

금속성과 거칠기 조합을 같은 조명에서 비교하고, 광원을 바꿔도 재질의 특성이 유지되는지 확인합니다.

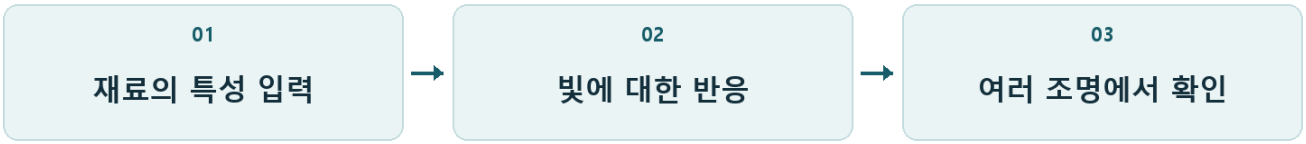


그림 07 / PBR과 재질 입력 개념도

원리

PBR은 빛과 재료의 상호작용을 물리적으로 그럴듯하게 근사하는 제작 방식입니다. Base Color는 기본 색, Metallic은 금속성, Roughness는 반사가 퍼지는 정도를 정합니다. Specular는 주로 비금속 표면의 반사 세기를 조절하는 입력입니다. 각 값의 역할을 나누면 조명이 바뀌어도 재질을 일관되게 만들 수 있습니다.

작동 과정

보통 비금속의 Base Color는 여러 방향으로 퍼지는 확산 반사 색에, 금속의 Base Color는 금속 반사의 색에 관여합니다. 거칠기가 높으면 반사가 넓게 퍼집니다. 순수 재료는 Metallic 0 또는 1을 기준으로 하고, 재료 경계나 아주 작은 혼합 패턴의 평균에는 중간값이 나타날 수 있습니다.

아트 실무

페인트와 녹은 비금속으로, 벗겨져 드러난 철은 금속으로 구분합니다. 조명 때문에 생긴 방향성 그림자나 하이라이트를 기본 색에 그려 넣으면 라이트가 움직일 때 충돌합니다. 재료에 남은 얼룩과 장면 조명에 의해 달라지는 음영을 구별해 제작합니다.

실무에서 비교하고 검증하기

성능 비용

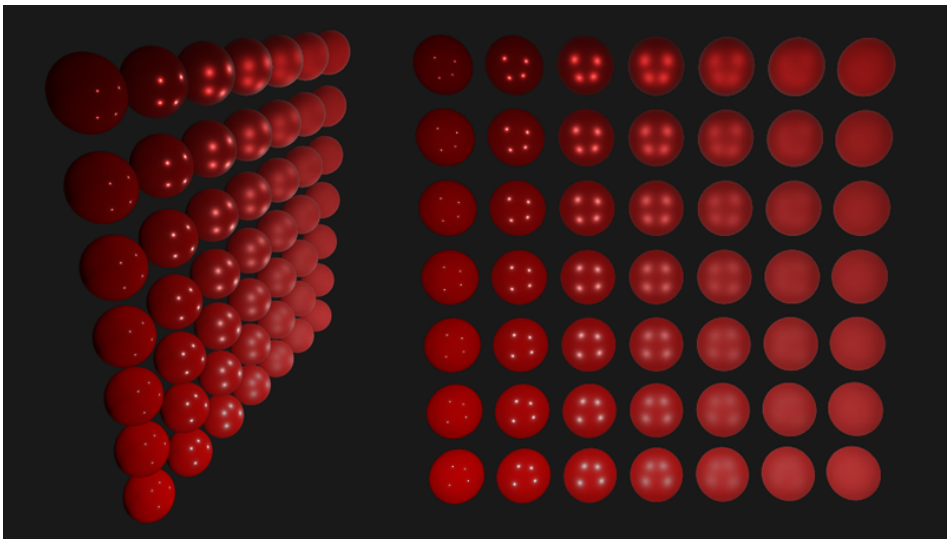
거칠기 숫자만 바꾸는 일과 텍스처를 추가해 복잡하게 혼합하는 일은 비용이 다릅니다. ORM 패킹은 가림·거칠기·금속성 데이터를 이미지의 R·G·B 채널 각각에 다른 숫자 정보를 넣는 방식입니다. 같은 좌표와 조건으로 읽을 수 있을 때 접근 횟수를 줄일 수 있습니다. 텍스처의 sRGB 옵션은 색 이미지용 변환을 적용해 읽는 설정입니다. 거칠기 같은 숫자 데이터에 이 변환을 적용하면 값이 달라질 수 있어 색 이미지와 구분합니다.

언리얼 실습

Material Instance는 원본 재질의 구조를 공유하면서 노출한 설정값을 바꾸는 변형입니다. 두 금속성 값과 두 거칠기 값의 조합을 만들고 같은 조명에서 비교합니다. 환경의 빛을 제공하는 Sky Light와 넓은 광원에서 반사를 봅니다. 광원 방향만 바꿔 기본 색에 고정된 음영이 남는지 확인합니다.

핵심 요약

재료 자체의 특성은 재질 입력으로 표현하고, 조명 변화에 따른 결과는 다른 조명에서도 검증합니다.



외부 렌더 이미지 · 금속성과 거칠기를 달리한 구들을 두 시점에서 비교합니다.

읽을 곳 오른쪽 배열의 가로 방향에서 거칠기가 변할 때 작은 반사점이 넓어지는 모습을 봅니다. 세로 방향의 금속성 변화도 함께 비교합니다. UE와 광원·톤매핑이 같은 비교 자료는 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · CC BY 4.0 · 출처 [40]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

공식 자료와 이어 읽기

[08] Physically Based Materials
[09] Using Fresnel
CH08 CH09 용어집 보충 주석

08 BRDF와 반사 분포

챕터 학습 목표

빛이 표면에서 어떤 방향으로 분포하는지 이해합니다. 하이라이트의 폭과 모양을 조정할 때 거칠기, 표면 방향, 광원 크기의 역할을 구분해 원하는 재질감을 설계합니다.

예제 학습 목표

구의 거칠기와 라이트 크기를 각각 따로 바꿔 반사 폭을 만드는 두 원인을 비교합니다.

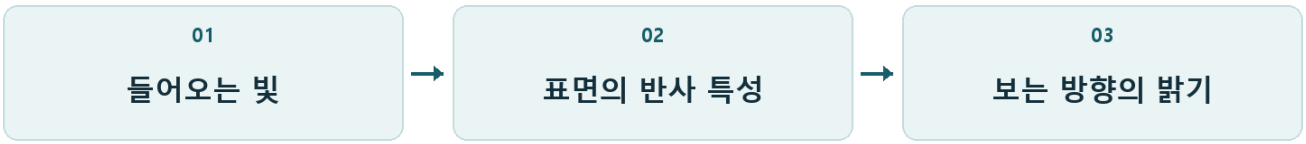


그림 08 / BRDF와 반사 분포 개념도

원리

BRDF는 어느 방향에서 온 빛이 어느 방향으로 얼마나 반사되는지를 나타내는 함수입니다. 함수는 입력 조건에 따라 결과를 정하는 계산 규칙이라는 뜻입니다. BRDF를 이해하면 같은 색의 물체도 거칠기와 관찰 방향에 따라 다르게 보이는 이유를 설명할 수 있습니다.

작동 과정

많은 재질 모델은 표면을 아주 작은 거울 조각들의 집합처럼 근사합니다. 이를 미세면 모델이라고 합니다. 조각의 방향이 고르게 모이면 날카로운 반사가, 넓게 흩어지면 퍼진 반사가 나타납니다. 반사율의 각도 변화와 조각 사이의 가림도 함께 반영합니다. 화면 밝기에는 광원의 밝기·크기와 실제 그림자도 영향을 줍니다.

아트 실무

돌 바닥의 닳은 길과 거친 가장자리에 같은 기본 색을 사용해도 거칠기 차이로 사용감을 표현할 수 있습니다. 반사광이 지나치게 넓다면 거칠기뿐 아니라 광원의 크기도 봅니다. 표면에서 밝게 반사되어 보이는 부분을 하이라이트라고 부릅니다.

실무에서 비교하고 검증하기

성능 비용

같은 셰이더에서 Roughness 값만 바꾸면 계산 구조는 그대로일 수 있습니다. 코팅과 바탕의 반사를 따로 계산하면 평가할 반사 성분이 늘 수 있습니다. 투과까지 포함하는 더 넓은 규칙은 BSDF라고 하며, Substrate에서는 이런 성분을 조합합니다. 수식의 항을 더 읽고 싶다면 보충 주석 2를 참고합니다.

언리얼 실습

같은 구에 거칠기 세 단계를 적용합니다. 면 형태의 광원인 Rect Light의 크기를 고정하고 하이라이트 폭을 비교합니다. 다음에는 재질을 고정하고 광원 크기만 바꿉니다. 두 실험을 분리해야 재질의 변화와 광원의 변화를 구분할 수 있습니다.

핵심 요약

반사의 모양은 재질과 빛의 조건이 함께 만듭니다. 수식을 외우기보다 각 입력이 화면을 어떻게 바꾸는지 연결합니다.



외부 개념도 · 미세한 표면 방향의 차이가 반사광의 퍼짐에 영향을 주는 모습입니다.

읽을 곳 거친 표면에서는 여러 방향으로, 매끈한 표면에서는 더 모인 방향으로 반사하는 차이를 봅니다. 확대된 개념도이며 실제 메시의 모든 미세 면을 직접 만드는 뜻은 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · CC BY 4.0 · 출처 [41]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

공식 자료와 이어 읽기

[08] Physically Based Materials
[10] Overview of Substrate Materials
[31] Real Shading in Unreal Engine 4 · Brian Karis
CH09 CH11 용어집 보충 주석

09 Fresnel과 시선 각도

챕터 학습 목표

시선 각도에 따라 반사가 달라지는 이유를 이해합니다. 물·유리·젖은 바닥·코팅 표면의 반사를 설계하고, 광학적 반응과 의도적으로 추가한 림 효과를 구분해 조절합니다.

예제 학습 목표

바닥을 보는 각도에 따른 반사 변화와 Fresnel 노드로 만든 발광 마스크를 각각 관찰합니다.

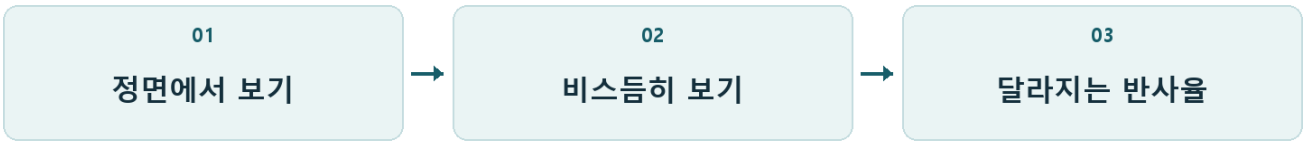


그림 09 / Fresnel과 시선 각도 개념도

원리

Fresnel은 빛이 두 물질의 경계를 만날 때 각도에 따라 반사 비율이 달라지는 성질입니다. 물이나 유리 같은 비금속은 정면보다 스치는 각도에서 반사가 강해지는 경우가 흔합니다. F0는 표면에 수직으로 빛이 들어올 때의 반사율을 뜻합니다.

작동 과정

잔잔한 물을 위에서 보면 물속이 잘 보이다가 낮은 시선에서는 하늘 반사가 강해지는 현상을 떠올릴 수 있습니다. 재질 모델은 각도에 따른 반사율을 계산하거나 근사합니다. 근사는 계산 비용을 줄이면서 주요 경향을 재현하는 방법입니다. 세부 수식보다 정면과 가장자리의 반응을 먼저 관찰합니다.

아트 실무

젖은 바닥을 높은 시점과 낮은 시점에서 비교합니다. UE의 Fresnel 노드는 시선과 표면 방향에 따라 달라지는 마스크에도 쓸 수 있습니다. 마스크는 효과를 어느 부분에 얼마나 적용할지 정하는 값입니다. 이를 발광에 연결한 림 효과는 재질의 자연 반사와 목적을 구분해 조절합니다.

실무에서 비교하고 검증하기

성능 비용

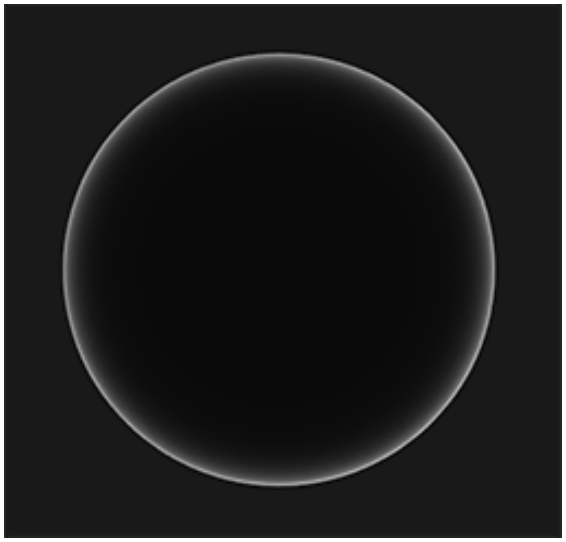
간단한 각도 마스크의 계산과 새 투명 레이어·반사 기능을 추가하는 일은 부담이 다릅니다. 마스크를 만드는 노드 수보다 그 값으로 어떤 추가 작업을 켰는지도 확인합니다. 실제 반사 계산의 각도와 아트용 Fresnel 마스크의 각도 사용은 구현이 다를 수 있습니다.

언리얼 실습

반사할 환경이 있는 레벨에서 젖은 판을 정면과 낮은 시선으로 봅니다. 별도 재질에서 Fresnel 출력을 Emissive에 연결해 마스크의 모양만 확인합니다. Emissive는 표면 자체가 밝게 보이도록 하는 입력입니다. 두 결과가 조명 변화에 어떻게 반응하는지 비교합니다.

핵심 요약

정면의 반사율과 각도에 따른 변화, 의도적으로 넣은 림 표현의 역할을 구분합니다.



외부 시각화 이미지 · 구의 중심과 실루엣 부근에서 Fresnel 반사 계수가 달라지는 예입니다.

읽을 곳 중심보다 테두리가 밝습니다. 곡면의 테두리에서 시선이 표면에 더 비스듬하게 닿기 때문입니다. 물체의 완성 렌더가 아닌 반사 계수의 시각화입니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · CC BY 4.0 · 출처 [41]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

공식 자료와 이어 읽기

[08] Physically Based Materials
[09] Using Fresnel
[10] Overview of Substrate Materials
[31] Real Shading in Unreal Engine 4 · Brian Karis
CH08 CH12 용어집 보충 주석

10 Shader와 순열과 WPO

챕터 학습 목표

재질과 움직임을 만드는 그래프가 GPU의 어떤 작업으로 이어지는지 이해합니다. 여러 에셋이 공유할 재질과 흔들리는 식생을 설계할 때 실행 부담, 준비할 프로그램의 종류와 배포 준비 시간을 구분해 관리합니다.

예제 학습 목표

같은 풀 메시에서 WPO를 켜고 끄며 변형 범위, 플랫폼 통계와 Base Pass·그림자 비용을 확인합니다.

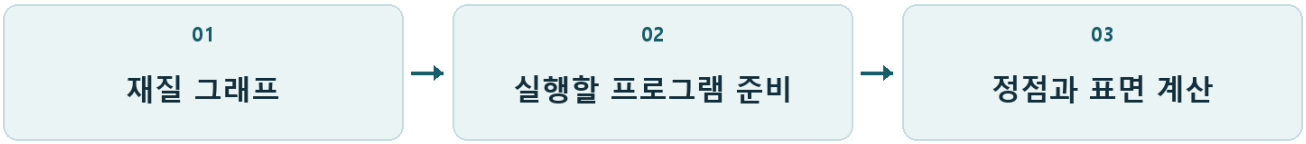


그림 10 / Shader와 순열과 WPO 개념도

원리

셰이더는 GPU에서 실행되는 계산 프로그램입니다. 재질 그래프는 표면의 색과 반사 등을 노드로 표현한 제작 화면이며, 엔진은 이를 GPU가 실행할 프로그램으로 바꿉니다. 이런 준비 과정을 컴파일이라고 합니다. 같은 그래프라도 플랫폼과 사용 기능에 따라 여러 실행 버전이 필요할 수 있습니다.

작동 과정

정점 셰이더는 모델 꼭짓점의 위치 등을 처리하고, 픽셀 셰이더는 화면에서 표면을 표현할 값을 계산합니다. WPO는 World Position Offset의 약자로 정점에 위치 변화를 더하는 입력입니다. 풀 끝에 시간에 따라 변하는 이동량을 주면 흔들림을 만들 수 있습니다. 표현할 굽힘의 세밀함은 원래 정점의 수와 배치에도 영향을 받습니다.

아트 실무

여러 재질이 공유할 기본 그래프를 마스터 재질이라고 부릅니다. Static Switch는 컴파일할 때 경로를 고르는 스위치입니다. 실제로 쓰는 스위치 조합에 따라 별도 버전, 즉 셰이더 순열이 생길 수 있습니다. 반면 일반 Scalar Parameter는 같은 실행 버전에서 숫자를 바꾸는 용도입니다.

실무에서 비교하고 검증하기

성능 비용

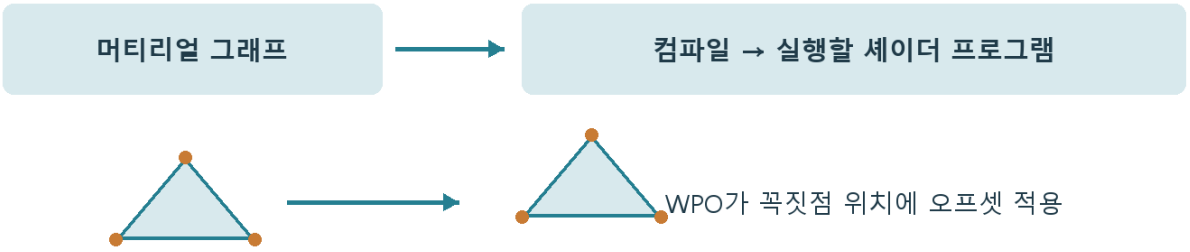
정점 수, 화면 면적, 텍스처 읽기와 계산이 각각 부담을 만듭니다. 텍스처 샘플은 이미지에서 필요한 좌표의 값을 읽는 작업입니다. 캐시는 자주 쓰는 데이터를 가까이 두어 다시 읽기 쉽게 하는 임시 저장 영역입니다. 컴파일 시간과 배포용 데이터를 준비하는 쿠킹 시간은 실행 중 GPU 시간과 따로 기록합니다.

언리얼 실습

같은 풀 메시에서 WPO 켜짐·꺼짐을 비교합니다. 변형한 정점이 오브젝트의 가시성 판정 범위인 bounds 밖으로 나가는지 봅니다. Material Editor의 Platform Stats는 대상 플랫폼용 셰이더 통계입니다. 이 통계와 함께 실제 Base Pass·Shadow Depths 시간을 기록합니다. 뒤의 이름은 각각 표면 준비와 그림자용 깊이 작업을 가리킵니다.

핵심 요약

노드 개수, 컴파일할 버전 수, 실제 프레임의 작업량은 서로 다른 지표입니다. 그래프의 변화가 어느 작업에 영향을 주는지 확인합니다.



정적 스위치는 실행 프로그램의 경우의 수에도 영향을 줍니다.

직접 제작 설명도 / 정적 분기는 컴파일할 프로그램의 경우의 수를, WPO는 정점 위치를 바꿉니다. 그래프 노드 수를 GPU 시간으로 바로 읽지 않습니다.

공식 자료와 이어 읽기

[02] Render Dependency Graph

[11] Material Editor UI

[12] Material Inputs

CH15 CH22 용어집 보충 주석

11 Substrate와 재질 레이어

챕터 학습 목표

물질의 혼합과 적층이 빛에 어떻게 다르게 반응하는지 이해합니다. 도장·코팅·물기 같은 복합 표면을 만들 때 필요한 표현을 선택하고, 대상 기기에서 허용할 계산과 저장량 안에서 품질을 조정합니다.

예제 학습 목표

단일 Slab, 두 층 적층과 파라미터 혼합을 비교해 외관·단순화 결과·패스 비용의 차이를 확인합니다.

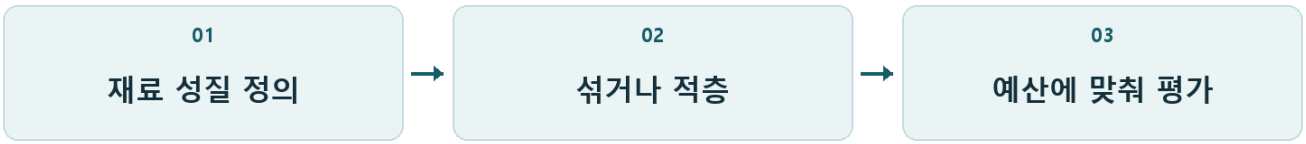


그림 11 / Substrate와 재질 레이어 개념도

원리

Substrate는 여러 재료의 빛 반응을 조합하는 머티리얼 제작 체계입니다. Slab BSDF는 재료 하나의 반사·투과·산란 성질을 나타내는 기본 단위입니다. 노드 이름의 Slab은 층을 떠올리게 하지만, 단일 재료를 정의하는 데도 사용합니다. 먼저 단일 재료가 어떻게 보이는지 확인한 뒤 혼합을 추가합니다.

작동 과정

Horizontal Blend는 페인트와 드러난 금속처럼 표면에서 나란히 존재하는 재료의 비중을 섞습니다. Vertical Layer는 코팅 아래에 나무가 있는 것처럼 위층과 아래층의 반응을 연결합니다. 두 재료의 설정값을 합쳐 하나의 재료로 근사하는 방식은 파라미터 블렌딩입니다. 복수 반응을 따로 계산하는 것과 외관·비용이 달라질 수 있습니다.

아트 실무

벗겨진 철문은 가로 혼합을, 물막이 있는 돌은 적층을 후보로 둡니다. Diffuse Albedo는 확산 반사의 색, F0는 정면 반사율, Roughness는 반사가 퍼지는 정도입니다. 넓은 바닥에서는 실제 적층과 거칠기만 조정한 단순 버전을 함께 만들어 표현 차이가 작업량을 정당화하는지 봅니다.

실무에서 비교하고 검증하기

성능 비용

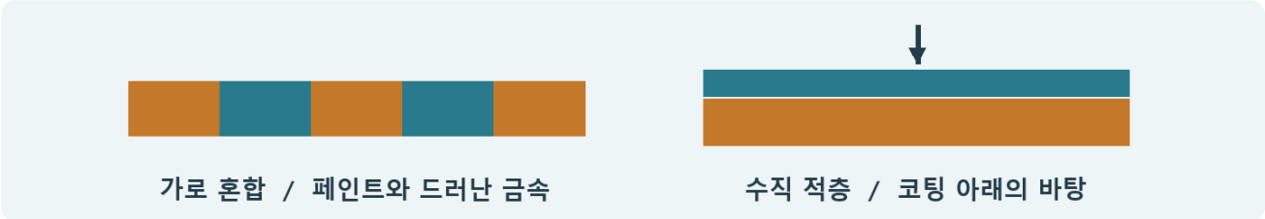
Closure는 실행 시 조명에 대해 평가할 재료 반응의 단위를 가리킵니다. 그래프의 Slab 수와 최종 평가 수가 항상 같지는 않습니다. Blendable GBuffer는 고정된 저장량과 예측 가능한 비용을 지향하며, Adaptive는 더 복잡한 반응을 담되 예산에 따라 단순화할 수 있습니다. 예산은 픽셀당 저장 바이트와 평가 수 등의 제한입니다. 지원 조건은 보충 주석 3에 정리했습니다.

언리얼 실습

기존 재질을 복제한 뒤 단일 Slab, 두 층 적층, 파라미터 혼합을 만듭니다. 화면에서 차지하는 면적을 맞춥니다. Material Editor와 Substrate 시각화에서 자동 단순화 여부를 확인하고 관련 GPU 시간을 비교합니다. GBuffer Format 변경은 프로젝트 설정이므로 맵만 바꿔 자동 전환되는 것으로 가정하지 않습니다.

핵심 요약

표현하려는 재료 관계에 맞춰 혼합 방식을 선택합니다. 보기 복잡한 그래프와 실제로 복잡하게 평가되는 재질을 구분합니다.



직접 제작 설명도 / 가로 혼합과 수직 적층은 다른 표면 구성을 나타냅니다. 층을 추가한 결과는 조명·거칠기·플랫폼 조건을 유지하고 검증합니다.

공식 자료와 이어 읽기

[05] Programming with Substrate GBuffer Formats

[06] Decal Materials

[10] Overview of Substrate Materials

CH08 CH13 용어집 보충 주석

12 Translucency와 굴절

챕터 학습 목표

배경을 투과시키는 표면의 셰이딩·정렬·굴절·합성을 이해합니다. 유리, 물, 연기와 겹친 효과에서 발생하는 화면 오류를 진단하고, 표현에 필요한 레이어와 화면 점유율을 판단합니다.

예제 학습 목표

유리판의 겹침과 굴절을 바꾸며 내부 배경, 화면 가장자리, 정렬 오류와 반투명 비용을 비교합니다.

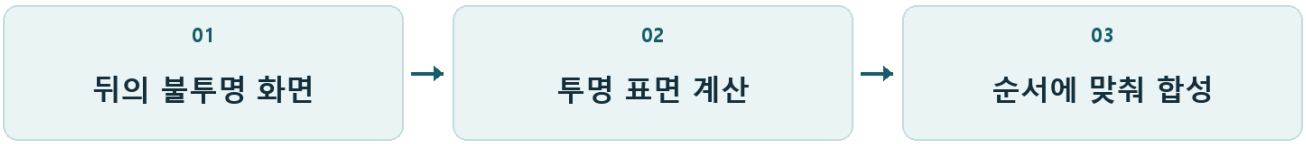


그림 12 / Translucency와 굴절 개념도

원리

Translucency는 뒤의 색이 비치는 표면을 표현합니다. 가장 단순한 알파 합성은 앞 색과 뒤 색을 투명도에 따라 섞는 방식입니다. 알파는 이때 사용할 비중을 나타내는 값입니다. 굴절은 물질 경계에서 빛의 방향이 바뀌어 배경이 휘어 보이는 현상입니다.

작동 과정

반투명은 보통 불투명 표면을 그린 뒤 별도 작업으로 합성합니다. 앞뒤 순서에 따라 섞인 색이 달라지므로 그리기 순서도 중요합니다. Sort Priority는 큰 합성 순서를 조절하는 설정이지만, 서로 교차한 두 면의 모든 화면 위치에서 올바른 앞뒤를 정해 주지는 못합니다.

아트 실무

유리, 김 서림과 빗방울을 여러 장 쌓기 전에 한 표면에서 표현할 수 있는 부분을 찾아봅니다. 화면 기반 굴절은 이미 그린 화면의 다른 위치를 읽어 왜곡하는 방식입니다. 화면 밖 정보나 앞 물체에 가려져 없던 배경을 충분히 얻기 어려우므로 가장자리와 얇은 프레임 주변을 확인합니다.

실무에서 비교하고 검증하기

성능 비용

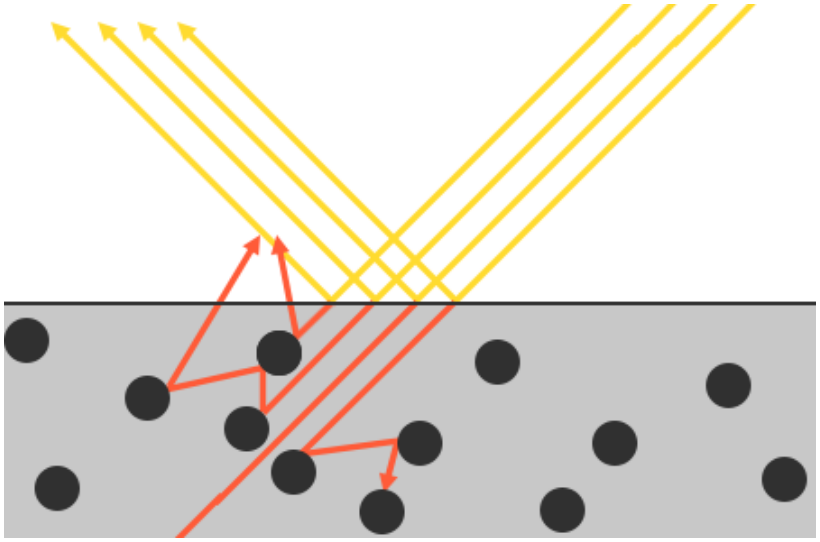
층 수, 화면을 덮는 면적, 조명 방식과 굴절이 모두 관여합니다. 그래프가 단순해도 큰 투명 레이어를 여러 장 합성하면 비용이 커질 수 있습니다. 불투명 깊이에 의해 가려지는 것과 투명층끼리 색을 합성하는 작업을 구분해 생각합니다.

언리얼 실습

같은 유리판을 한 장과 네 장으로 배치합니다. 카메라·조명·재질을 맞추고 겹침만 비교합니다. 다음에는 굴절 켜짐·꺼짐만 바꿉니다. 카메라를 옆으로 움직여 정렬과 가장자리 오류를 보고 Translucency 관련 GPU 시간을 기록합니다.

핵심 요약

반투명 문제는 재질 입력, 합성 순서와 읽을 수 있는 배경 정보로 나누어 조사합니다.



외부 개념도 · 입사한 빛이 표면에서 반사되거나 내부로 들어가 산란·흡수될 수 있습니다.

읽을 곳 표면에서 되돌아가는 경로와 내부로 들어가는 경로를 구분합니다. 이 개념만으로 UE 반투명 구현이 물질 내부의 모든 광선을 추적한다고 해석하지 않습니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · CC BY 4.0 · 출처 [41]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

공식 자료와 이어 읽기

[13] Lit Translucency
[14] Using Refraction
CH05 CH23 용어집 보충 주석

13 SSS와 표면 아래 산란

챕터 학습 목표

표면 아래를 지난 빛이 만드는 외관과 재질 두께의 역할을 이해합니다. 왁스·대리석·피부·식생의 밀도감과 역광 표현에 맞는 산란 모델과 거리 스케일을 선택합니다.

예제 학습 목표

두께가 다른 왁스 형상을 전면광·역광에서 비교하고 일반 표면과 산란 재질의 결과를 확인합니다.

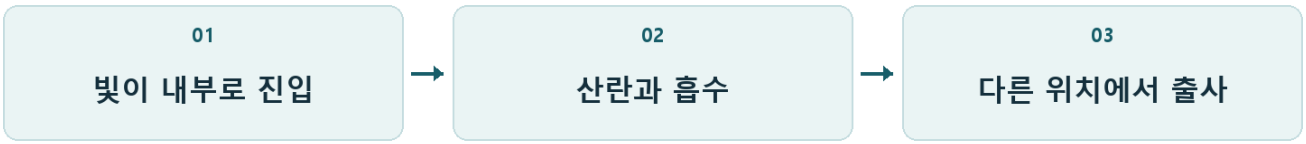


그림 13 / SSS와 표면 아래 산란 개념도

원리

SSS는 Subsurface Scattering의 약자로, 빛이 표면 아래에서 산란하는 현상입니다. 왁스에 들어간 빛이 내부에서 방향을 바꾸고 일부는 흡수된 뒤 다른 위치로 나와 부드러운 외관을 만듭니다. 단순히 뒤 배경이 보이는 투명도와 구별해 관찰합니다.

작동 과정

UE의 Subsurface Profile은 화면에 보이는 정보를 이용해 산란의 퍼짐을 근사하는 방식입니다. Profile은 퍼짐과 색 반응을 묶은 설정입니다. Substrate의 SSS MFP는 내부에서 빛이 상호작용하기까지의 평균 거리 스케일입니다. 같은 값이라도 물체의 두께와 크기, 사용하는 모델에 따라 결과가 달라집니다.

아트 실무

촛농의 얇은 가장자리와 두꺼운 몸통을 역광에서 비교합니다. 작은 소품을 거대한 크기로 가져오면 거리 기반 산란 설정이 의도와 다르게 보일 수 있습니다. 잎 뒷면의 빛 표현에 쓰는 Two Sided Foliage와 왁스용 산란 설정은 표현 목적과 처리 방식에 맞춰 선택합니다.

실무에서 비교하고 검증하기

성능 비용

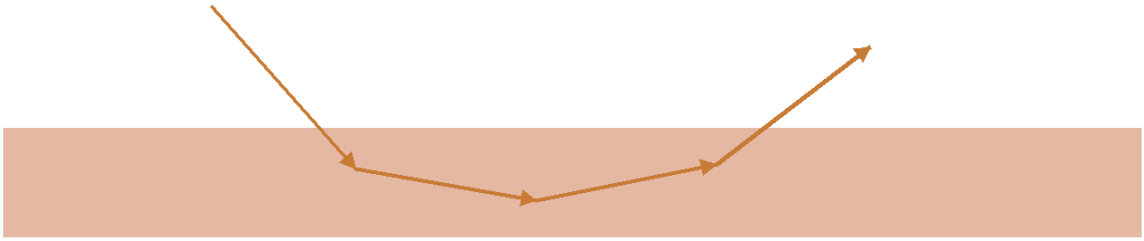
일반적인 표면 반사에 더해 산란을 근사하는 작업이 필요할 수 있습니다. 화면을 얼마나 채우는지와 사용하는 구현 경로가 영향을 줍니다. SSS라는 이름만으로 같은 비용이나 같은 투과 결과를 예상하지 않습니다.

언리얼 실습

실제 크기에 맞춘 왁스 기둥과 얇은 판을 만듭니다. 기본 조명 반응 모델인 Default Lit와 Subsurface Profile 또는 Substrate SSS를 각각 적용합니다. 노출을 고정한 전면광·역광에서 두께에 따른 차이를 보고, 화면 면적을 맞춰 GPU 작업도 비교합니다.

핵심 요약

산란 모델, 거리 스케일과 물체 두께를 함께 조정해 내부를 지난 빛의 외관을 만듭니다.



표면 안쪽 산란의 개념 / UE 구현의 실제 광선 경로가 아님

직접 제작 설명도 / 빛이 표면 안으로 들어가 여러 방향으로 퍼져 나온 상황을 단면으로 표현했습니다. UE의 화면 기반 근사는 실제 내부 광선을 모두 추적하는 방식과 다릅니다.

공식 자료와 이어 읽기

[10] Overview of Substrate Materials

[15] Subsurface Profile Shading Model

CH07 CH11 용어집 보충 주석

14 Draw Call과 배칭

챕터 학습 목표

메시 섹션과 렌더 상태가 패스별 그리기 명령으로 이어지는 방식을 이해합니다. 모듈 배경과 소품의 슬롯·아틀라스·합체 단위를 설계할 때 CPU 제출 비용과 가시성 손실을 함께 판단합니다.

예제 학습 목표

형상이 같은 다중 섹션 메시와 단일 섹션 메시를 비교해 호출 수와 CPU·GPU 시간이 어떻게 달라지는지 확인합니다.

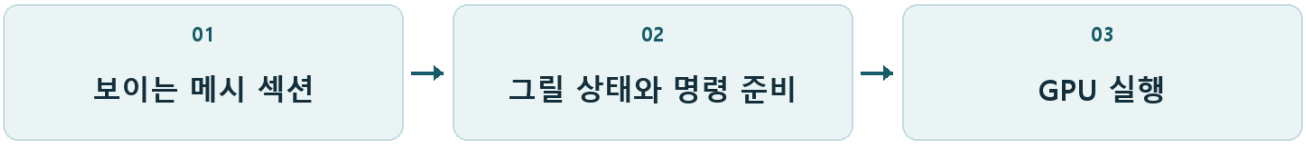


그림 14 / Draw Call과 배칭 개념도

원리

Draw Call은 GPU에 특정 지오메트리를 그리도록 제출하는 명령입니다. 하나의 Actor에 메시 하나가 있어도, 재질이 나뉜 섹션이나 깊이·그림자 패스 때문에 여러 그리기 작업이 생길 수 있습니다. Actor는 레벨에 배치한 오브젝트의 단위이므로 그 개수와 호출 수를 같게 세지 않습니다.

작동 과정

메시 섹션은 보통 서로 다른 재질을 적용할 수 있는 메시의 구역입니다. 렌더러는 보이는 섹션에 필요한 재질과 그리기 상태를 준비합니다. 호환되는 작업을 묶어 제출 부담을 줄이는 것을 배칭이라고 합니다. 실제로 묶일 수 있는 조건은 메시·재질·패스와 엔진 처리 방식에 따라 달라집니다.

아트 실무

소품 하나에 나무·금속·흙 슬롯을 각각 두면 표현은 편하지만 그리기 작업이 늘 수 있습니다. 여러 이미지를 한 장에 모으는 텍스처 아틀라스와 공유 재질을 대안으로 검토합니다. 반대로 동네 전체를 한 메시로 합치면 일부만 보일 때 필요한 부분을 제외하기 어려워질 수 있습니다.

실무에서 비교하고 검증하기

성능 비용

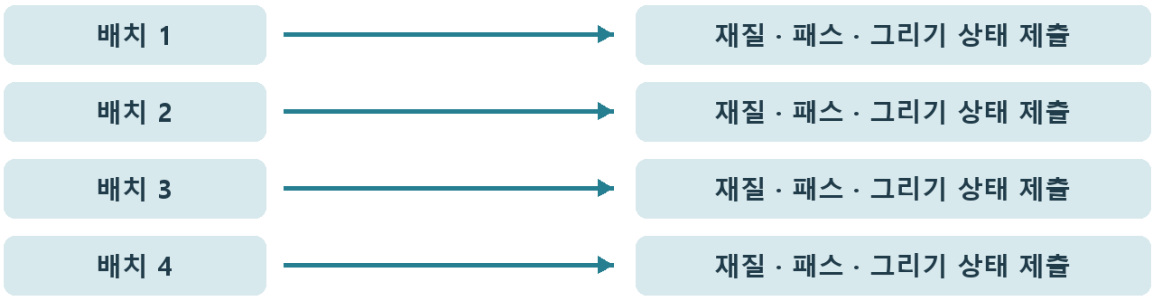
명령과 상태를 준비하는 일은 CPU 부담이 될 수 있습니다. 호출을 줄여도 GPU가 처리하는 표면 면적과 재질 계산이 비슷하면 GPU 시간은 거의 그대로일 수 있습니다. CPU 제출이 병목인 상황과 화면 계산이 병목인 상황을 나누어 판단합니다.

언리얼 실습

형상과 외관이 비슷한 세 섹션 메시와 한 섹션 메시를 만듭니다. 같은 수를 같은 구도로 배치합니다. stat scenerendering 또는 stat rhi에서 관련 그리기 수치를 보고, stat unit의 Draw와 GPU 시간을 기록합니다. 여기서 Draw는 주로 CPU 렌더 스레드 시간이며 Draw Call 개수를 뜻하는 항목은 아닙니다.

핵심 요약

오브젝트 개수, 메시 섹션과 패스별 호출, CPU·GPU 시간을 각각 확인합니다.



직접 제작 설명도 / 서로 다른 그리기 상태를 CPU에서 준비하고 GPU에 제출합니다. Actor 하나가 Draw Call 하나라는 뜻은 아니며 여러 패스와 재질 슬롯이 영향을 줍니다.

공식 자료와 이어 읽기

[03] Introduction to Performance Profiling and Configuration

[16] Mesh Drawing Pipeline

CH15 CH25 용어집 보충 주석

15 Instancing과 반복 배치

챕터 학습 목표

반복 배치에서 공유 가능한 데이터와 인스턴스마다 필요한 작업을 구분합니다. 울타리·식생·군집 소품을 만들 때 ISM·HISM과 개별 배치의 관리·가시성·갱신 조건을 판단합니다.

예제 학습 목표

같은 기둥 200개를 개별 Actor와 ISM으로 배치하고 전부 또는 일부가 보이는 조건에서 비용을 비교합니다.

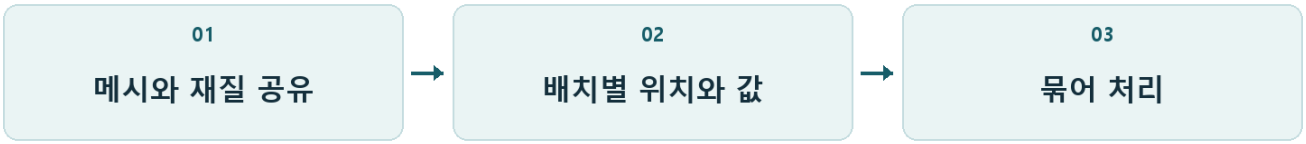


그림 15 / Instancing과 반복 배치 개념도

원리

인스턴싱은 같은 메시 데이터를 여러 배치에서 재사용하는 방식입니다. 울타리 기둥마다 별도의 메시 데이터를 만드는 대신, 공통 형상과 재질을 공유하고 각 기둥의 위치·회전·크기를 따로 보관합니다. 이런 위치와 방향·크기의 정보를 변환 또는 Transform이라고 합니다.

작동 과정

ISM은 Instanced Static Mesh Component의 약자로 반복 메시를 한 컴포넌트 안에서 관리합니다. 컴포넌트는 Actor가 가진 메시·조명 같은 기능 요소입니다. HISM은 여기에 공간적으로 묶은 계층을 사용합니다. 어느 묶음을 보이게 하고 어떤 세부 형상을 쓸지는 기능과 버전의 처리 방식을 확인합니다.

아트 실무

같은 기둥 200개에 조금씩 다른 색을 주고 싶다면 Per Instance Custom Data를 검토합니다. 각 배치에 추가 숫자를 전달하고 재질에서 읽는 기능입니다. 메시와 재질을 매번 새로 만들지 않고 변주할 수 있습니다. 충돌과 그림자, 갱신할 위치의 수 같은 실제 사용 조건도 함께 맞춥니다.

실무에서 비교하고 검증하기

성능 비용

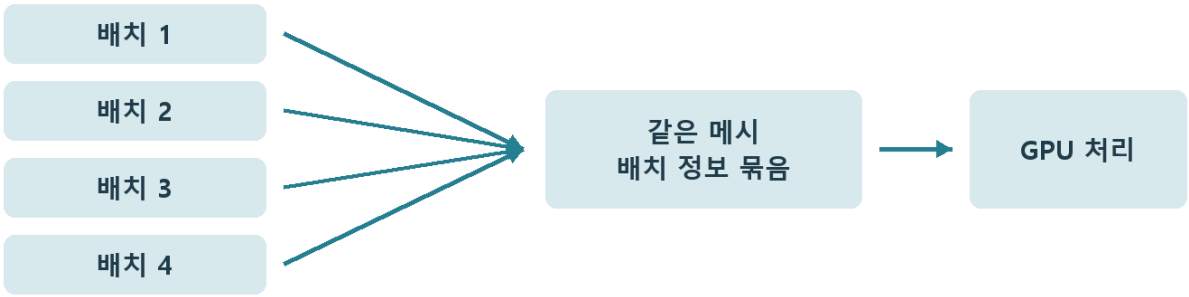
배치마다 Actor와 컴포넌트를 관리하는 부담, 그리기 제출 부담을 줄일 수 있습니다. 공유해도 실제로 보이는 기둥의 삼각형과 픽셀 계산은 남습니다. 모든 기둥이 고정된 경우와 매 프레임 위치를 바꾸는 경우의 갱신 비용도 다를 수 있습니다.

언리얼 실습

개별 Static Mesh Actor 200개와 ISM 안의 200개를 별도 레벨 변형으로 준비합니다. 모든 기둥이 보이는 구도와 일부만 보이는 구도에서 Draw·GPU 시간을 기록합니다. 필요하면 HISM도 비교합니다. 어떤 방식을 쓰든 수량·재질·충돌·그림자 조건을 맞춥니다.

핵심 요약

공유로 줄이는 관리·제출 비용과 실제 가시 지오메트리의 계산량을 분리합니다.



직접 제작 설명도 / 같은 메시의 배치를 묶어 제출 준비를 줄일 수 있습니다. 각 배치의 화면 크기·재질·그림자 계산 비용은 여전히 남습니다.

공식 자료와 이어 읽기

[16] Mesh Drawing Pipeline

[17] Instanced Static Mesh Component

CH14 CH18 용어집 보충 주석

16 LOD와 화면 크기

챕터 학습 목표

화면에서 읽히는 정보에 맞춰 디테일을 배분합니다. 원거리 에셋의 실루엣·그림자·재질을 유지하면서 지오메트리와 섹션을 줄이고, FOV와 플랫폼에 맞는 전환 기준을 정합니다.

예제 학습 목표

난간의 LOD별 형상과 화면 크기를 확인하고 카메라 이동 중 실루엣·그림자 전환과 비용을 비교합니다.

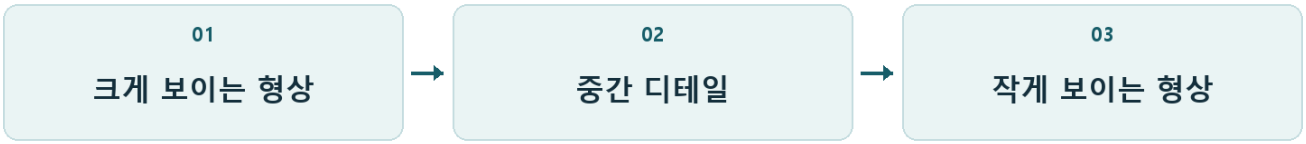


그림 16 / LOD와 화면 크기 개념도

원리

LOD는 Level of Detail의 약자로 화면에서 필요한 정도에 맞춰 디테일을 바꾸는 방식입니다. 가까운 난간에는 나사와 베벨이 보이지만 멀리서는 기둥의 굵기와 윤곽이 더 중요합니다. 전통적인 Static Mesh LOD는 여러 단계의 대체 메시를 사용합니다.

작동 과정

UE는 화면에서 차지하는 크기를 기준으로 LOD를 선택할 수 있습니다. 그래서 같은 거리라도 메시의 크기나 카메라의 시야각이 바뀌면 선택 결과가 달라질 수 있습니다. 시야각은 FOV라고 부르며, 넓을수록 더 넓은 범위를 한 화면에 담습니다. 단계별로 삼각형·섹션·재질을 줄이는 방법을 검토합니다.

아트 실무

작은 나사와 베벨을 먼저 줄이고 멀리서도 읽히는 상단 윤곽과 기둥 간격은 유지합니다. 단순화 과정에서 노멀과 색이 달라져 번쩍이는 전환이 생기는지도 봅니다. 디더 전환은 작은 점무늬의 가시성을 바꾸며 두 단계를 이어 보이게 하는 방법이며, 이동 중의 안정성까지 확인합니다.

실무에서 비교하고 검증하기

성능 비용

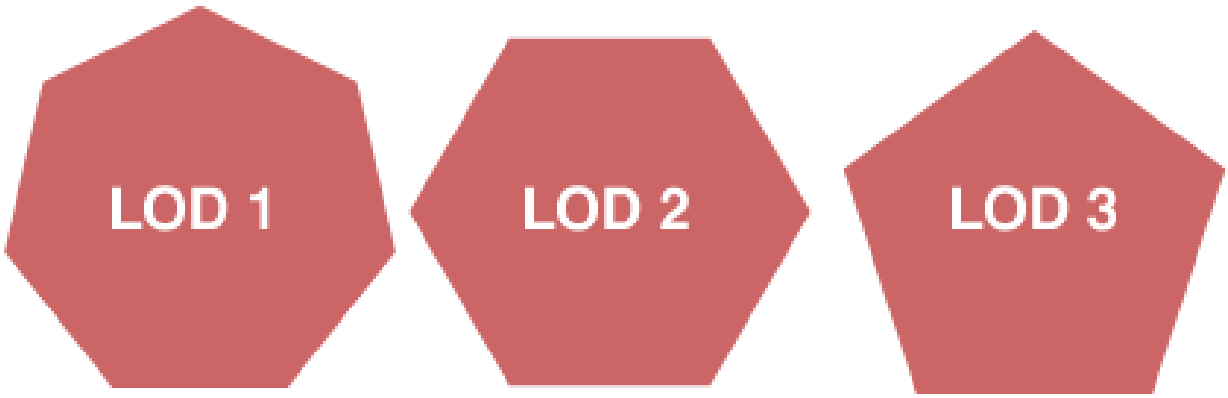
정점 처리와 그림자에 그릴 지오메트리를 줄일 수 있습니다. 하지만 섹션 수와 화면 면적, 재질이 그대로라면 호출과 픽셀 비용은 많이 줄지 않을 수 있습니다. 전환 중 두 표현을 처리하는 구간의 비용도 함께 측정합니다.

언리얼 실습

Static Mesh Editor에서 각 LOD의 형상과 Screen Size를 확인합니다. Screen Size는 화면 크기 기준입니다. 지원되는 LOD Coloration으로 선택 단계를 보고 Forced LOD로 특정 단계를 고정해 형상을 비교합니다. 같은 카메라 이동에서 윤곽·그림자 튐과 GPU 시간을 기록합니다.

핵심 요약

멀리서 읽히는 정보를 유지하면서 줄일 대상을 고릅니다. 거리 하나보다 화면 크기와 실제 효과를 기준으로 판단합니다.



외부 비교 그림 · 비슷한 실루엣을 더 적은 변으로 근사한 세 가지 LOD입니다.

읽을 곳 오른쪽으로 갈수록 위쪽 윤곽이 각지게 바뀝니다. 원본의 LOD 1·2·3 표기는 해당 그림의 번호이며 UE에서 흔히 시작하는 LOD0 번호와 구별합니다.

Sirotk · Wikimedia Commons · CC BY-SA 3.0 · 출처 [42]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

공식 자료와 이어 읽기

[17] Instanced Static Mesh Component
[18] Optimizing LOD Screen Size Per Platform
CH17 CH23 용어집 보충 주석

17 Nanite와 가상화 지오메트리

챕터 학습 목표

가상화 지오메트리가 디테일과 가시 데이터를 선택하는 원리를 이해합니다. 고밀도 바위·건축·식생에 Nanite를 적용할 때 적합성, 겹침, 재질과 스트리밍 비용을 함께 판단합니다.

예제 학습 목표

같은 절벽에서 Nanite와 전통 LOD를 비교하고 클러스터·오버드로·이동 중 팝인과 비용을 확인합니다.

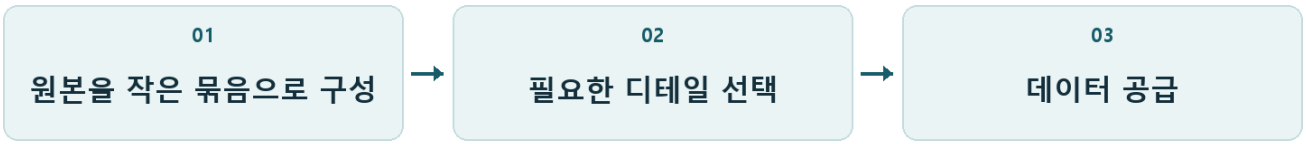


그림 17 / Nanite와 가상화 지오메트리 개념도

원리

Nanite는 높은 디테일의 메시를 효율적으로 표시하기 위한 가상화 지오메트리 시스템입니다. 가상화는 원본의 모든 데이터를 항상 같은 방식으로 처리하는 대신, 필요한 부분과 세부 수준을 선택해 공급하는 접근입니다. 엔진은 메시를 작은 삼각형 묶음인 클러스터와 그 계층으로 구성합니다.

작동 과정

화면에서 허용할 형상 오차와 가시성을 바탕으로 클러스터를 선택합니다. 원본과 표시 형상이 화면에서 얼마나 달라도 되는지를 생각하면 오차 기준을 이해하기 쉽습니다. 필요한 메시 데이터는 스트리밍으로 공급합니다. Nanite는 전용 가시성·래스터 경로를 사용하므로 전통 메시의 호출 방식과 그대로 대응시키지 않습니다.

아트 실무

복잡한 절벽에는 좋은 후보가 되지만 얇은 잎이 촘촘히 겹친 수관도 별도로 시험합니다. 서로 가까운 표면이 깊이상 겹치면 화면 계산이 남을 수 있습니다. 다른 렌더링 기능이 원본 대신 참조할 수 있는 대체 표현을 폴백 메시라고 합니다. 그 품질과 대상 플랫폼의 지원도 확인합니다.

실무에서 비교하고 검증하기

성능 비용

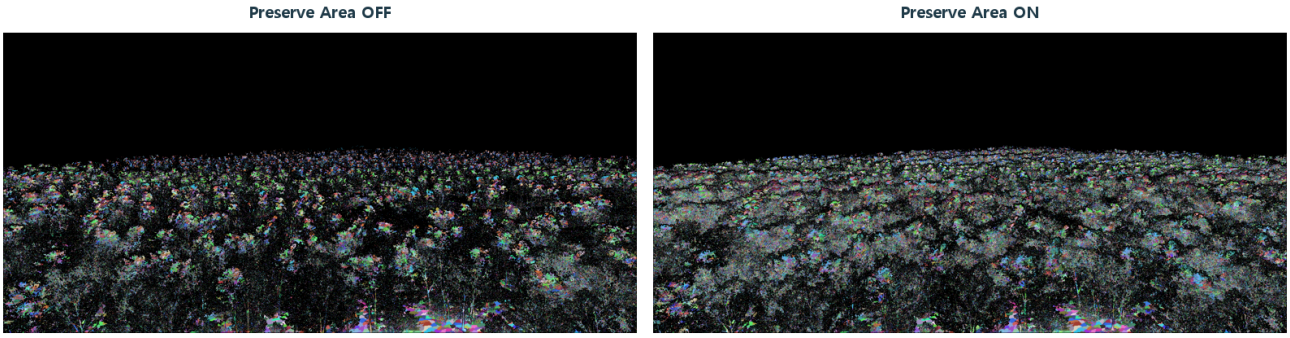
원본 폴리곤 수 외에 인스턴스 수, 겹친 클러스터, 구멍 마스크와 정점 변형 재질, 디스크에서 데이터를 읽는 작업이 관여합니다. Nanite를 켠 사실만으로 조명·그림자·재질 비용이 없어지지는 않습니다. 기능 지원은 사용하는 엔진 버전과 프로젝트 조건을 기준으로 확인합니다.

언리얼 실습

같은 절벽 배치에 Nanite와 전통 LOD 버전을 준비합니다. Nanite Visualization에서 Triangles는 표시 삼각형, Clusters는 처리 묶음, Overdraw는 겹침을 살피는 데 사용합니다. 근거리·원거리·빠른 이동에서 외관과 GPU 시간을 기록합니다. 폴백 표현 때문에 비교 품질이 달라지지 않았는지도 봅니다.

핵심 요약

Nanite의 이점과 남아 있는 재질·겹침·스트리밍 비용을 함께 검증합니다.



Epic 공식 비교 이미지 · Nanite Clusters 시각화에서 Preserve Area를 끈 상태와 켠 상태입니다.

읽을 곳 왼쪽의 검게 비어 보이는 잎 사이 공간과 오른쪽의 더 조밀한 수관을 비교합니다. 삼각형을 줄일 때 소실되는 잎 면적을 보존하는 사례입니다. 색은 재질색이 아닌 클러스터 구분이며 UE 5.5 문서의 예시로, 이 프로젝트의 UE 5.8 실측이나 성능 비교가 아닙니다.

© Epic Games · UE 5.5 공식 문서 · 공식 문서 참고 이미지 · 출처 [45]
원본 2점 나란히 배치 · 상단 표제 추가 · 색상 유지 · 원저작권은 Epic Games에 귀속

공식 자료와 이어 읽기

[17] Instanced Static Mesh Component
[19] Nanite Virtualized Geometry
CH06 CH20 용어집 보충 주석

18 Culling과 가시성

챕터 학습 목표

화면에 기여하지 않는 작업을 줄이는 가시성 판정과 bounds의 역할을 이해합니다. 건물·실내·변형 에셋의 분할 단위를 정하고, 불필요한 렌더링과 갑작스러운 사라짐을 진단합니다.

예제 학습 목표

꺾인 복도의 합체·분할 메시에서 bounds, 가림 판정과 모서리 이동 중의 표시 변화를 비교합니다.

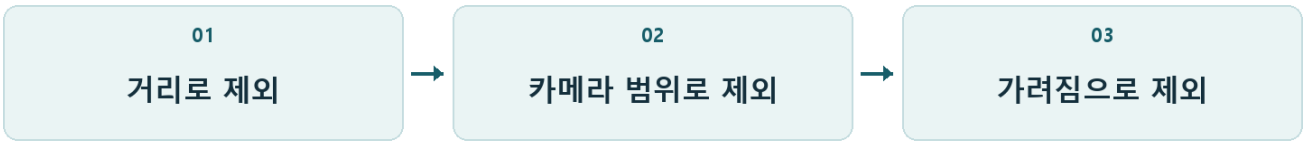


그림 18 / Culling과 가시성 개념도

원리

컬링은 최종 화면에 필요하지 않은 렌더링 후보를 제외하는 과정입니다. 멀리 있는 물체, 카메라 밖 물체, 다른 물체 뒤에 완전히 가린 물체를 각각 다른 기준으로 판정합니다. 화면의 가시성을 잘 나누면 이후 단계에서 처리할 메시와 표면 계산을 줄일 수 있습니다.

작동 과정

카메라가 볼 수 있는 3D 공간을 뷰 프러스텀 또는 시야 절두체라고 합니다. 엔진은 먼저 범위 밖 후보를 줄이고 깊이 정보 등으로 가려짐을 판정할 수 있습니다. 물체의 모든 삼각형 대신 물체를 감싸는 상자나 구를 사용하는 경우가 많습니다. 이 둘러싼 범위가 bounds입니다.

아트 실무

긴 건물 전체를 하나로 합치면 작은 부분만 보여도 큰 bounds가 가시 판정을 받을 수 있습니다. 반대로 공간 단위로 나누면 벽 뒤 부분을 제외하기 쉬워질 수 있습니다. WPO로 흔들리는 풀은 원래 bounds 밖으로 정점이 나가 갑자기 사라지는지도 봅니다.

실무에서 비교하고 검증하기

성능 비용

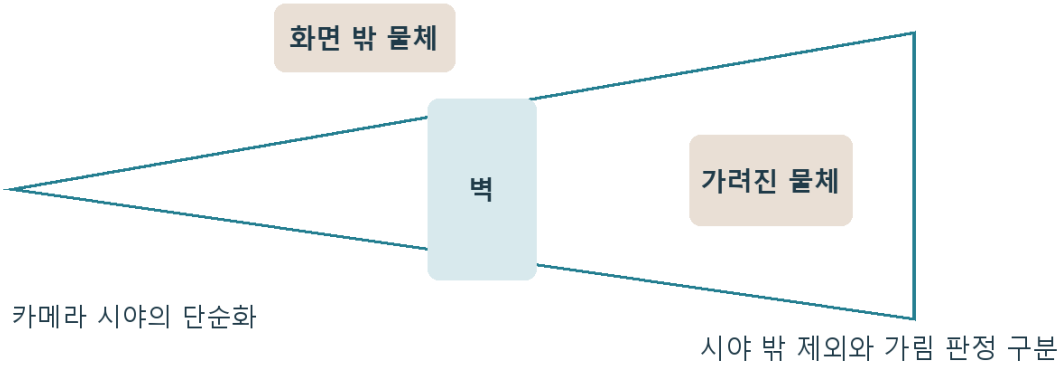
잘게 나누면 제외 기회가 늘 수 있지만 관리할 오브젝트와 가시성 검사·제출 작업도 늘어납니다. 큰 bounds는 너무 많은 부분을 가시 후보로 남길 수 있습니다. 메인 카메라에서 안 보이는 물체가 화면의 그림자에는 기여할 수 있다는 점도 구분합니다.

언리얼 실습

꺾인 복도에 합체 벽과 구획별 벽을 준비합니다. Viewport의 Bounds 표시로 판정 범위를 보고, 지원되는 경로에서 r.VisualizeOccludedPrimitives 1로 가려진 후보를 확인합니다. 카메라를 모서리 너머로 움직여 표시 수치와 갑작스러운 등장·사라짐을 기록합니다.

핵심 요약

작업 모듈의 편의와 가시성 판정의 효율을 함께 고려해 메시를 나눕니다.



직접 제작 설명도 / 화면 밖 물체와 앞 물체에 가려진 물체는 서로 다른 이유로 제외 후보가 됩니다. 실제 가시성 판정에는 보수적 경계와 프레임 지연이 개입할 수 있습니다.

공식 자료와 이어 읽기

[17] Instanced Static Mesh Component

[19] Nanite Virtualized Geometry

[30] Visibility and Occlusion Culling

CH05 CH14 용어집 보충 주석

19 Mipmap과 텍스처 샘플링

챕터 학습 목표

한 화면 픽셀이 덮는 텍스처 범위와 필터링을 이해합니다. 먼 바닥·비스듬한 벽·미세 노멀의 깜빡임과 흐림을 진단하고, 밉·필터·원본 패턴을 품질과 대역폭 관점에서 조정합니다.

예제 학습 목표

같은 복도 바닥에서 밉 사용 여부와 개별 밉을 비교하고 이동 중 패턴의 안정성을 확인합니다.

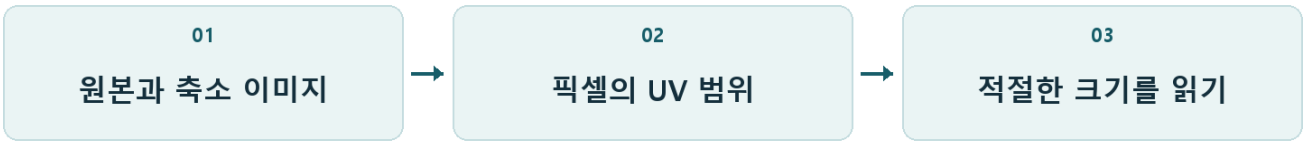


그림 19 / Mipmap과 텍스처 샘플링 개념도

원리

텍스처 샘플링은 이미지에서 필요한 위치의 값을 읽는 작업입니다. 모델 표면의 어느 지점이 이미지의 어디에 해당하는지 정하는 2D 좌표를 UV라고 합니다. 물체가 멀어지면 화면 한 픽셀이 텍스처의 넓은 영역을 덮게 되므로 원본 한 점만 읽어서는 안정적인 결과를 얻기 어렵습니다.

작동 과정

밉맵은 원본을 점점 작게 축소해 준비한 이미지 계층입니다. 엔진은 화면에서 UV가 얼마나 빠르게 바뀌는지 등을 이용해 읽을 단계를 고릅니다. 여러 값을 섞어 적절한 결과를 만드는 처리를 필터링이라고 합니다. 비스듬한 바닥처럼 한 방향으로 길게 늘어진 영역에는 이방성 필터링이 도움을 줄 수 있습니다.

아트 실무

복도 바닥을 낮은 시선으로 보며 줄눈과 미세 노멀의 깜빡임을 확인합니다. 너무 작은 반복무늬는 움직일 때 보였다 사라질 수 있습니다. 밉과 필터만 조절하기보다 원본 패턴의 크기, 노멀의 강도와 재질 거칠기도 함께 살핍니다.

실무에서 비교하고 검증하기

성능 비용

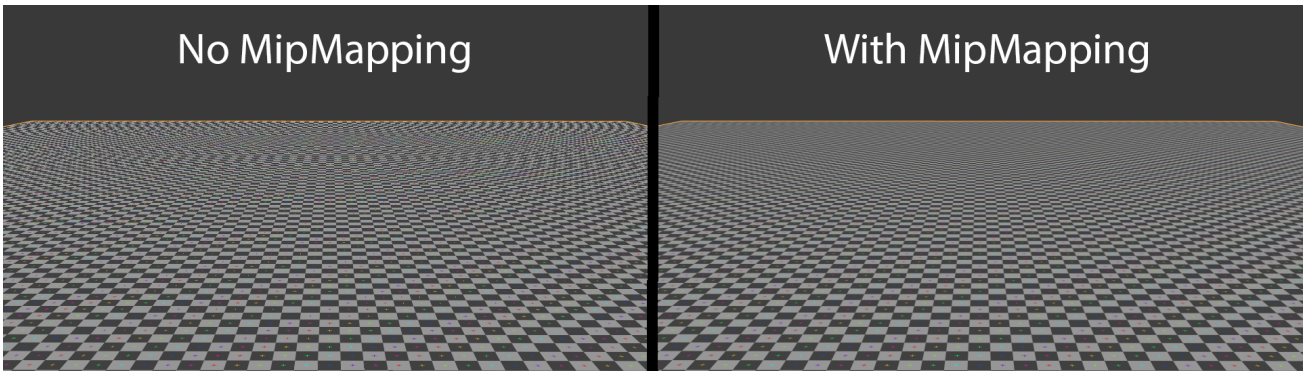
이상적인 정사각형 2D 텍스처의 완전한 밉 체인은 최상위 단계보다 약 1/3의 텍셀을 더 가집니다. 텍셀은 텍스처의 한 데이터 칸입니다. 실제 저장량은 압축과 배치 방식에 따라 달라집니다. 필요한 작은 밉을 읽으면 캐시와 대역폭 사용에 유리할 수 있습니다.

언리얼 실습

원본 텍스처를 복제해 일반 밉과 NoMipmaps를 비교합니다. Texture Editor에서 개별 밉이 어떤 패턴을 남기는지 확인합니다. 같은 카메라 이동에서 바닥의 선명도와 깜빡임을 기록합니다. 메모리에 원하는 밉이 실제로 올라왔는지는 스트리밍 상태도 함께 확인합니다.

핵심 요약

밉은 작게 보이는 텍스처를 안정적으로 읽기 위한 준비입니다. 밉 선택과 메모리 상주 관리는 서로 연결된 다른 작업입니다.



외부 실제 렌더 비교 · 왼쪽 No MipMapping, 오른쪽 With MipMapping입니다.

읽을 곳 수평선에 가까운 먼 바닥에서 왼쪽의 물결무늬와 오른쪽의 안정된 패턴을 비교합니다. 원본은 Blender 렌더이며 필터링도 관여할 수 있으므로 UE에서 밉만 바꾼 통제 실험으로 해석하지 않습니다.

BillyBob CornCob · Wikimedia Commons · CC0 1.0 · 출처 [43]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 이용 조건

공식 자료와 이어 읽기

[20] Texture Asset Editor
[21] Texture Streaming Overview
CH20 CH23 용어집 보충 주석

20 Texture Streaming과 메모리 예산

챕터 학습 목표

화면의 텍스처 수요와 메모리에 상주하는 데이터의 차이를 이해합니다. 큰 환경을 이동할 때 흐린 재질·팝인·끊김이 예산, 수요 추정, I/O 중 무엇과 연결되는지 판단합니다.

> **예제 학습 목표**
>
> 시장 씬의 첫 진입과 재방문을 비교하고 일부 소품의 해상도를 낮춰 풀 압박과 외관 변화를 확인합니다.

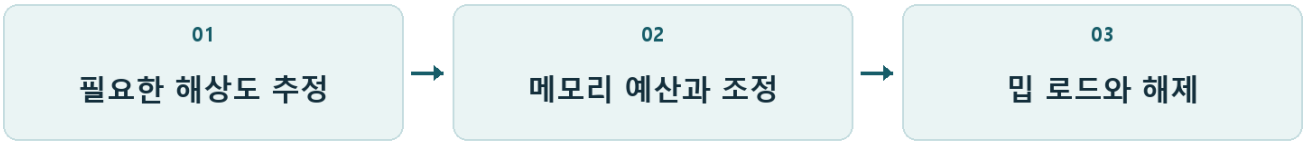


그림 20 / Texture Streaming과 메모리 예산 개념도

원리

텍스처 스트리밍은 필요에 따라 텍스처의 해상도 단계를 메모리로 가져오거나 내보내는 관리 방식입니다. 화면에서 원하는 밉과 지금 메모리에 있는 밉은 다를 수 있습니다. 일반 Texture Streaming, 텍스처를 작은 타일로 나누는 Virtual Texturing, 월드의 Actor를 불러오는 방식은 구분해서 읽습니다.

작동 과정

스트리머는 카메라와 물체의 거리, 화면에서 필요한 텍셀 밀도 등을 이용해 수요를 추정합니다. 텍셀 밀도는 표면이나 화면 범위에 어느 정도의 텍스처 정보가 필요한지와 관련된 크기 기준입니다. 이후 메모리 예산과 우선순위에 맞춰 로드·해제를 요청합니다. 디스크에서 데이터를 옮기는 작업을 I/O라고 부릅니다.

아트 실무

시장 전경의 모든 소품에 큰 텍스처를 쓰면 한 번에 필요한 메모리가 커질 수 있습니다. 가까이 볼 간판과 먼 배경 용기를 나눠 해상도를 정합니다. 흐리게 보이면 텍스처 크기뿐 아니라 UV 스케일, 수요 추정과 로딩 지연도 확인합니다.

실무에서 비교하고 검증하기

성능 비용

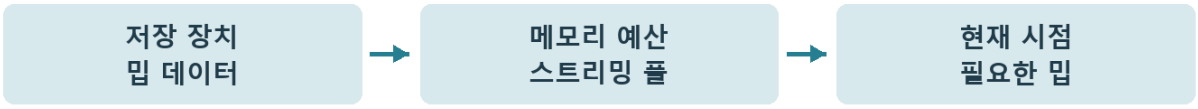
스트리밍 풀은 이 관리에 사용할 메모리 예산의 한 묶음입니다. 풀을 키우면 높은 밉을 유지하기 쉬울 수 있지만 다른 리소스가 쓸 여유는 줄어듭니다. 큰 상주 데이터와 급격한 카메라 이동은 전송량·로딩 지연을 늘릴 수 있습니다. 경고 하나만 보고 무조건 풀부터 키우지 않습니다.

언리얼 실습

stat streaming을 켜고 같은 경로의 첫 진입과 재방문을 비교합니다. 처음 데이터를 읽는 조건을 콜드, 이미 준비된 조건을 웜이라고 부릅니다. 일부 소품의 최대 해상도만 낮추고 풀 사용량·선명도·갑자기 선명해지는 팝인을 기록합니다. 변경한 배치를 반영하도록 텍스처 스트리밍 데이터도 갱신합니다.

핵심 요약

메모리 부족, 잘못 추정한 수요와 데이터를 늦게 읽는 문제를 나누어 진단합니다.



읽기 지연과 예산 부족을 나누어 진단

직접 제작 설명도 / 시점에 필요한 밉을 저장 장치에서 메모리로 가져옵니다. 메모리 예산 부족과 읽기 지연은 다른 문제이므로 함께 기록합니다.

공식 자료와 이어 읽기

[20] Texture Asset Editor

[21] Texture Streaming Overview

[22] Texture Streaming Metrics

CH19 CH25 용어집 보충 주석

21 Lumen과 동적 간접광

챕터 학습 목표

간접광과 반사가 장면의 표현·추적·갱신에 의존하는 방식을 이해합니다. 실내 누광, 누락된 반사와 노이즈를 진단하고, 메시 구성·라이팅·추적 품질을 목표 성능에 맞게 설계합니다.

예제 학습 목표

L자 방의 Lumen Scene과 Surface Cache를 관찰하고 메시 분할 또는 추적 설정의 영향을 비교합니다.

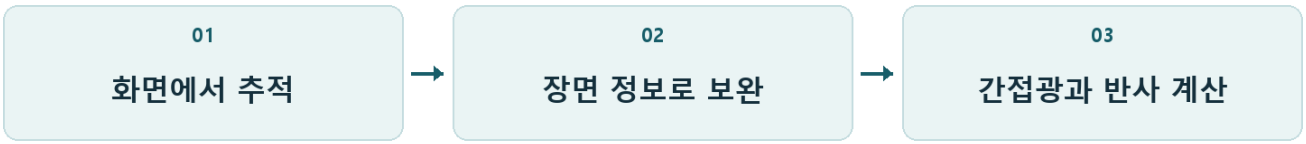


그림 21 / Lumen과 동적 간접광 개념도

원리

직접광은 광원에서 표면으로 바로 도달한 빛이고, 간접광은 다른 표면에서 반사되어 온 빛입니다. Lumen은 움직이는 장면의 간접광과 반사를 계산합니다. 빨간 벽에서 튄 빛이 흰 벽에 붉은 기운을 주는 현상은 전역 조명, 즉 장면 전체의 빛 전달을 이해하는 사례입니다.

작동 과정

Lumen은 먼저 화면에 있는 정보를 따라 빛의 경로를 조사하는 Screen Trace를 활용합니다. 부족한 부분은 거리장으로 표현한 장면이나 하드웨어 레이 트레이싱으로 보완할 수 있습니다. 거리장은 표면까지의 거리를 저장한 표현입니다. 레이 트레이싱은 광선과 물체가 만나는 위치를 추적하는 계산 방식입니다.

아트 실무

Surface Cache는 추적한 표면의 재질과 조명 정보를 빠르게 조회하기 위한 저장 체계입니다. 여러 방향에서 표면을 담은 Cards를 사용합니다. 한 메시로 된 방의 일부가 분홍색으로 표시되면 이 표면 정보의 적용 범위를 확인합니다. 벽·바닥·천장을 분리한 버전과 비교하고, 얇은 벽의 누광은 거리장 표현도 살핍니다.

실무에서 비교하고 검증하기

성능 비용

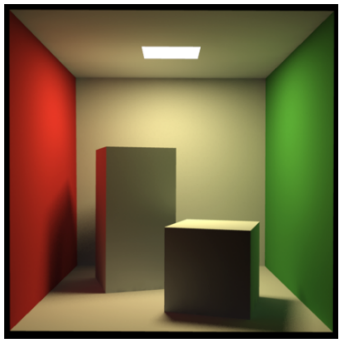
내부 해상도, 추적 거리, 반사 범위와 장면 갱신이 비용에 영향을 줍니다. 하드웨어 레이 트레이싱은 지원 GPU가 필요하며, 광선이 만날 후보를 빠르게 찾기 위한 가속 구조도 관리합니다. 인스턴스가 많거나 물체가 자주 변형되면 그 구조를 갱신하는 부담을 봅니다.

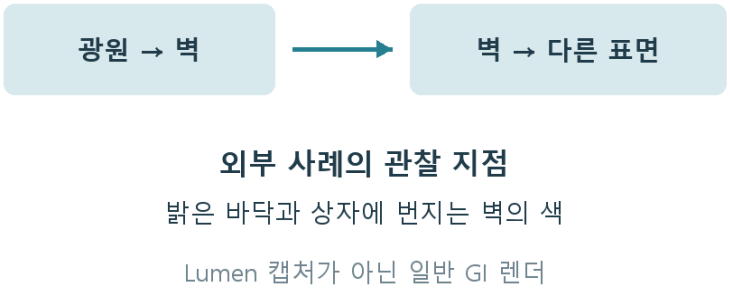
언리얼 실습

L자 방에서 노출을 고정합니다. Lumen Overview는 관련 결과를 모아 보는 모드, Lumen Scene은 추적에 쓰는 장면 표현, Surface Cache는 표면 정보의 적용 상태를 살피는 모드입니다. 메시 분할 또는 추적 설정 한 가지만 바꾸며 누광·반사·간접광과 GPU 시간을 비교합니다.

핵심 요약

현재 화면의 정보, 추적할 장면 표현과 계산 예산을 나누어 Lumen 문제를 조사합니다.





직접 제작 설명도 / 벽에서 반사된 빛이 다른 표면의 색에 기여하는 간접광입니다. 왼쪽 외부 렌더는 일반적인 사례이며 Lumen 실행 결과나 성능 비교가 아닙니다.

이미지 출처 · Trevorgoodchild · Public domain · 원본 색상과 비율 유지 [36]

공식 자료와 이어 읽기

[23] Lumen Technical Details

[24] Lumen Performance Guide

CH03 CH22 용어집 보충 주석

22 Virtual Shadow Maps와 페이지 캐시

챕터 학습 목표

그림자 해상도와 페이지 캐시 재사용이 연결되는 방식을 이해합니다. 넓은 환경과 움직이는 식생에서 라이트 범위·캐스터·변형이 그림자 품질과 갱신 비용에 미치는 영향을 판단합니다.

예제 학습 목표

풀밭의 바람과 카메라 이동 조건을 나눠 캐시 페이지 변화와 그림자 생성·투영 비용을 비교합니다.

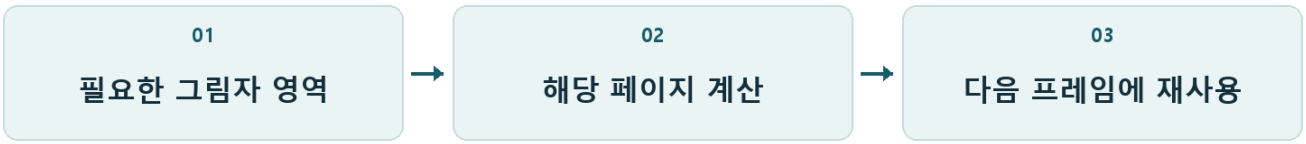


그림 22 / Virtual Shadow Maps와 페이지 캐시 개념도

원리

그림자 맵은 라이트에서 본 표면 깊이를 저장해 빛이 가려지는지 판정하는 자료입니다. VSM은 높은 해상도의 그림자 맵을 작은 페이지 단위로 제공하는 방식입니다. 필요한 부분만 계산하고 메모리에 유지해 넓은 장면의 세밀한 그림자를 다룹니다.

작동 과정

카메라 화면에 보이는 표면이 요구하는 그림자 영역을 찾고 해당 페이지를 준비합니다. 방향광에는 카메라 주변의 여러 범위를 계층으로 배치하는 클립맵을 사용합니다. 이전에 계산한 페이지를 다시 쓰는 것이 캐시 재사용입니다. 라이트나 물체가 변하면 과거 결과가 맞지 않아 페이지를 새로 계산할 수 있습니다.

아트 실무

그림자를 만드는 물체를 캐스터라고 부릅니다. 바람으로 정점을 움직이는 WPO 식생은 멀리서도 캐시 갱신을 일으킬 수 있습니다. 원거리 바람, 그림자 거리와 라이트 영향 반경을 각각 조정한 버전을 비교합니다. 한 번에 여러 변수를 끄면 무엇이 비용을 줄였는지 알기 어렵습니다.

실무에서 비교하고 검증하기

성능 비용

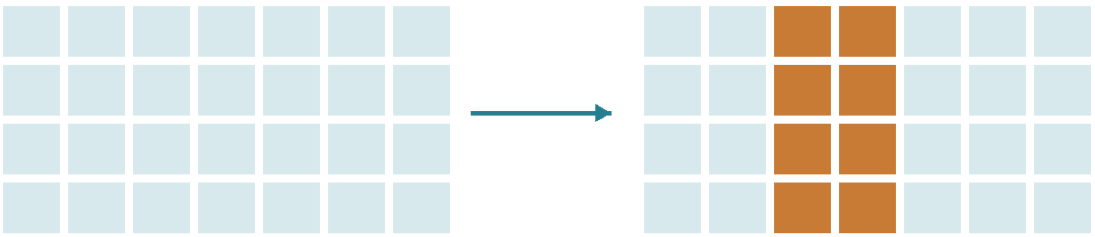
그림자용 깊이를 만드는 작업과, 그 결과를 표면에 적용하는 투영 작업은 별도 비용입니다. 새로 필요한 페이지, 움직이는 캐스터, 큰 bounds와 겹치는 광원은 작업량을 늘릴 수 있습니다. 캐시 무효화는 저장한 결과가 더 이상 맞지 않아 다시 계산해야 하는 상태를 뜻합니다.

언리얼 실습

Shadow Map Method에서 Virtual Shadow Maps를 확인합니다. 카메라를 고정하고 VSM Cached Page 시각화로 바람 켜짐·꺼짐을 비교합니다. 다음에는 바람 조건을 고정하고 카메라를 움직입니다. Shadow Depths와 Projection 계열 시간을 나누어 기록합니다.

핵심 요약

그림자 선명도와 함께, 이미 계산한 결과를 얼마나 재사용하는지 확인합니다.



직접 제작 설명도 / 그림자 캐시에 남아 있는 페이지는 재사용할 수 있지만, 움직임·변형 등에 의해 무효화된 영역은 다시 그려야 할 수 있습니다.

공식 자료와 이어 읽기

[19] Nanite Virtualized Geometry

[25] Virtual Shadow Maps

CH10 CH21 용어집 보충 주석

23 TAA와 프레임 히스토리

챕터 학습 목표

여러 프레임의 정보를 사용하는 화질 개선과 그 한계를 이해합니다. 철망·식생·반사처럼 얇거나 움직이는 디테일에서 깜빡임·번짐·잔상을 진단하고 모션 정보와 재질을 조정합니다.

예제 학습 목표

철망과 WPO 식생을 정지·이동 중 관찰하고 모션 벡터 및 잔상·깜빡임의 변화를 확인합니다.

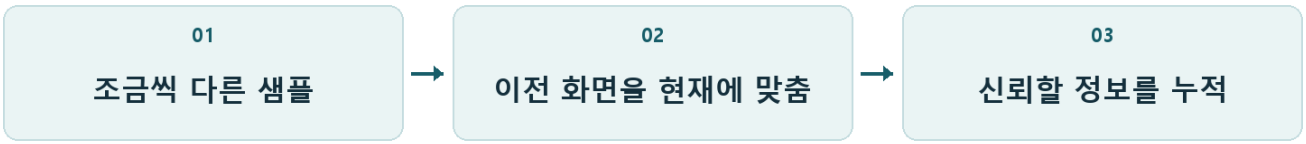


그림 23 / TAA와 프레임 히스토리 개념도

원리

TAA는 Temporal Anti-Aliasing의 약자로 여러 프레임의 정보를 사용해 계단과 깜빡임을 줄입니다. 한 번 그린 프레임에 없는 세부 정보를 다른 프레임의 샘플로 보완하는 접근입니다. 저장해 둔 이전 프레임의 정보를 히스토리라고 합니다.

작동 과정

카메라 투영 위치를 아주 조금씩 바꾸는 지터로 서로 다른 서브픽셀 위치를 샘플합니다. 서브픽셀은 한 픽셀보다 작은 위치 차이를 뜻합니다. 화면에서 표면이 얼마나 이동했는지를 나타내는 모션 벡터로 과거 정보를 현재 위치에 맞춥니다. 이 과정을 재투영이라고 합니다. 새로 드러난 표면처럼 믿기 어려운 과거 정보는 제한합니다.

아트 실무

철망과 케이블, 흔들리는 풀을 정지 화면과 이동 화면에서 비교합니다. 과거 정보가 잘못 남아 뒤로 끌리는 자국은 고스팅이라고 부릅니다. 정점은 움직이는데 모션 정보가 그 움직임을 충분히 반영하지 못하면 시간적 처리에서 문제가 나타날 수 있습니다.

실무에서 비교하고 검증하기

성능 비용

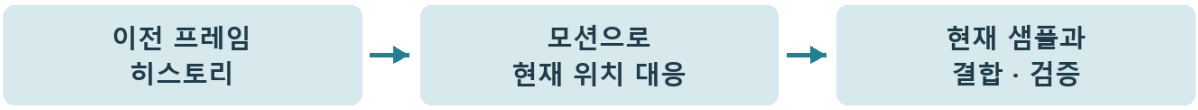
히스토리를 저장하는 메모리와 현재 화면에 맞추는 계산이 필요합니다. 오래 누적하면 안정성이 좋아질 수 있지만 움직임에 대한 응답과 선명도가 달라질 수 있습니다. 어떤 값을 조정할지는 정지 선명도보다 실제 이동 중의 결과와 함께 판단합니다.

언리얼 실습

Anti-Aliasing Method를 Temporal AA로 선택하고 내부 해상도를 100%로 시작합니다. 철망 앞을 같은 속도로 이동한 영상과 정지 화면을 기록합니다. WPO 켜짐·꺼짐을 비교하고, 지원되는 Velocity 시각화에서 모션 정보가 나타나는지 봅니다. Velocity는 이 문맥에서 화면 이동 정보와 연결되는 이름입니다.

핵심 요약

시간적 처리는 현재 화면과 과거 정보의 연결에 의존합니다. 정지 이미지 하나로 움직임 품질을 판정하지 않습니다.



가려짐 해제 · 얇은 선 · 이동 중 잔상을 함께 확인

직접 제작 설명도 / 이전 프레임 정보를 현재 위치로 옮겨 현재 샘플과 결합합니다. 가려짐 해제나 잘못된 모션 정보가 있으면 이전 값을 그대로 재사용하기 어렵습니다.

공식 자료와 이어 읽기

[26] Anti-Aliasing and Upscaling

[27] Temporal Super Resolution

CH10 CH24 용어집 보충 주석

24 TSR과 시간적 업스케일링

챕터 학습 목표

내부 렌더링 해상도와 최종 출력 해상도를 구분하고 시간적 재구성의 역할을 이해합니다. 목표 기기에서 이동 중 디테일의 안정성과 절약되는 GPU 작업을 함께 평가해 해상도 예산을 정합니다.

예제 학습 목표

출력 해상도를 고정하고 Screen Percentage 100과 67에서 글자·케이블·반사의 이동 품질과 패스 시간을 비교합니다.

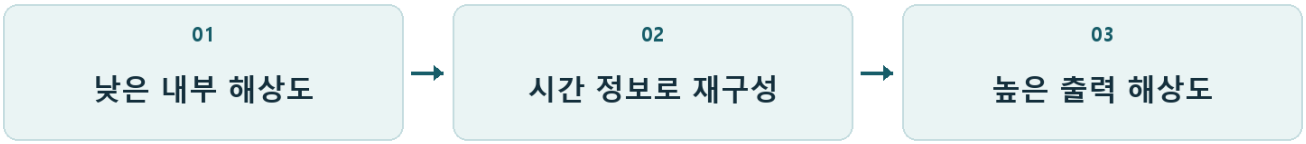


그림 24 / TSR과 시간적 업스케일링 개념도

원리

내부 해상도는 장면의 주요 계산을 수행하는 크기이고, 출력 해상도는 최종 표시할 화면 크기입니다. TSR은 Temporal Super Resolution의 약자로 낮은 내부 해상도와 여러 프레임의 정보를 이용해 높은 출력 해상도를 재구성하는 UE 업스케일러입니다.

작동 과정

낮은 해상도로 장면을 그리고 지터·깊이·모션과 히스토리를 사용해 더 촘촘한 출력을 만듭니다. 모션은 화면 이동 정보, 히스토리는 이전 프레임 정보입니다. 두 개념과 재투영은 CH23에서 설명했습니다. 새로 드러난 영역이나 움직임이 큰 부분에서는 과거 정보의 기여를 조절해야 합니다.

아트 실무

출력 크기를 같게 두고 간판 글자, 얇은 케이블과 젖은 벽 반사를 봅니다. 정지 뒤에는 선명해져도 움직이는 동안 정보가 뭉개질 수 있습니다. 게임 중 자주 보는 시점과 카메라 이동 속도에서 읽히는지를 기준으로 품질을 정합니다.

실무에서 비교하고 검증하기

성능 비용

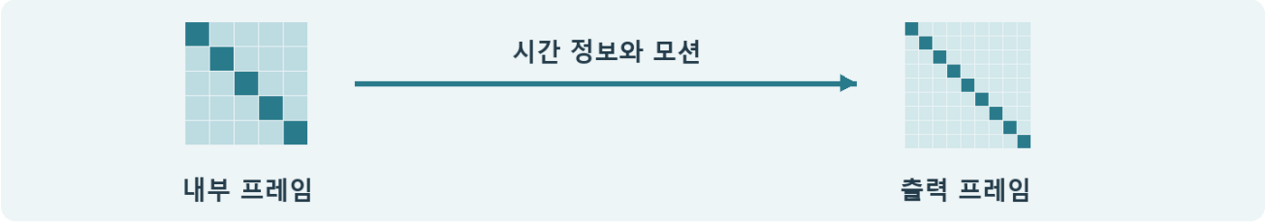
Screen Percentage는 출력 크기에 대한 내부 해상도의 가로·세로 비율을 정하는 설정입니다. 67%면 두 축이 각각 줄어 내부 픽셀 수는 약 45%입니다. 전체 GPU 시간까지 그 비율로 줄지는 않습니다. TSR 자체의 재구성과 출력 처리, CPU와 일부 그림자 작업은 별도로 남기 때문입니다.

언리얼 실습

Anti-Aliasing Method를 TSR로 선택합니다. 부하에 따라 내부 크기를 자동 변경하는 Dynamic Resolution을 끄고 출력 해상도를 고정합니다. r.ScreenPercentage 100과 67을 비교해 같은 이동 구간의 글자·철망·반사와 전체 GPU·TSR 관련 시간을 기록합니다.

핵심 요약

같은 출력에서 움직임 품질을 비교하고, 줄어든 장면 계산과 재구성 비용을 함께 측정합니다.



낮은 내부 해상도 → 시간 정보와 모션으로 출력 재구성

직접 제작 설명도 / 낮은 내부 해상도의 현재 정보에 모션·이전 프레임을 결합해 출력 영상을 재구성합니다. 그림은 처리 관계이며 품질 보장이나 실측 확대 결과가 아닙니다.

공식 자료와 이어 읽기

[26] Anti-Aliasing and Upscaling

[27] Temporal Super Resolution

CH23 CH25 용어집 보충 주석

25 CPU와 GPU 병목

챕터 학습 목표

프레임 처리율을 제한하는 원인과 스레드·GPU의 대기를 구분합니다. 에셋 수정이 필요한 상황과 게임 로직·제출·동기화 문제를 판별해 효과가 있는 최적화 작업의 우선순위를 정합니다.

예제 학습 목표

고정 경로에서 stat unit과 해상도 변경 반응을 확인한 뒤 Insights의 CPU·GPU·대기 구간을 대조합니다.

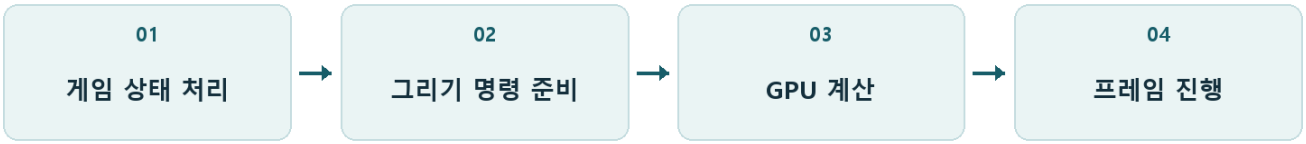


그림 25 / CPU와 GPU 병목 개념도

원리

병목은 해당 상황에서 처리 속도를 제한하는 작업입니다. CPU와 GPU는 서로 다른 프레임의 일을 겹쳐 처리할 수 있으므로 각 시간을 모두 더해 최종 프레임 시간으로 읽지 않습니다. 한쪽이 다른 쪽을 기다리면 표시된 시간이 실제 계산 시간과 다르게 보일 수도 있습니다.

작동 과정

stat unit의 Game은 주로 게임 로직을 처리하는 CPU 실행 흐름, Draw는 렌더 명령을 준비하는 흐름, GPU는 그래픽 계산 시간을 보는 출발점입니다. RHIT가 표시되면 그래픽 API 명령 처리 쪽의 CPU 흐름을 함께 봅니다. 스레드·RHI의 뜻은 CH01과 용어집에서 다시 확인할 수 있습니다.

아트 실무

내부 해상도를 낮춰 GPU 시간과 프레임 시간이 함께 줄면 화면 계산을 조사할 근거가 됩니다. 프레임 시간이 그대로라면 CPU, 대기와 프레임 제한도 봅니다. VSync는 화면 갱신에 출력 시점을 맞추는 기능이며, 설정한 FPS 제한과 함께 측정값을 일정 수준에 묶어 둘 수 있습니다.

실무에서 비교하고 검증하기

성능 비용

60 FPS의 프레임 간격은 약 16.67 ms, 30 FPS는 약 33.33 ms입니다. ms는 1초의 천분의 1입니다. 평균만 보면 드문 끊김을 놓칠 수 있습니다. P95 프레임 시간은 느린 쪽 5%가 시작되는 경계 부근을 보는 지표입니다. 최대 시간과 느린 구간의 원인도 함께 살핍니다.

언리얼 실습

성능 도구를 쓸 수 있는 Development 빌드를 목표 기기에서 실행합니다. stat unit·stat unitgraph로 경향을 보고 내부 해상도만 낮춰 반응을 확인합니다. Unreal Insights는 CPU·GPU 작업을 시간축에 펼쳐 보여 주는 분석 도구입니다. 느린 프레임에서 계산과 대기 구간을 대조합니다.

핵심 요약

프레임 제한과 대기를 확인한 뒤, 실제로 처리율을 제한하는 작업을 고릅니다.



직접 제작 설명도 / CPU와 GPU의 작업은 겹칠 수 있습니다. 단순화한 시간축에서 처리율을 제한하는 구간을 보며, 각 시간의 합을 전체 프레임 시간으로 사용하지 않습니다.

공식 자료와 이어 읽기

[03] Introduction to Performance Profiling and Configuration

[28] Stat Commands

CH14 CH26 용어집 보충 주석

26 GPU Profiler와 전후 검증

챕터 학습 목표

느리다는 인상을 재현 가능한 측정과 원인 후보로 바꾸는 방법을 익힙니다. 무거운 패스를 찾고 한 변수의 수정이 총 프레임 시간과 이미지 품질에 실제로 도움이 되는지 검증해 팀에 전달합니다.

예제 학습 목표

동일한 카메라 경로에서 한 항목만 바꿔 반복 측정하고 패스 시간·전체 프레임 시간·이동 화질을 전후 비교합니다.

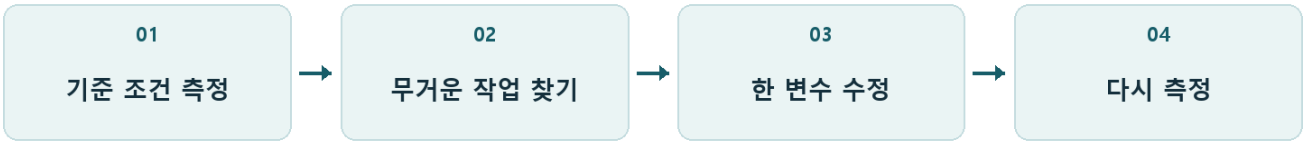


그림 26 / GPU Profiler와 전후 검증 개념도

원리

프로파일링은 실행 중 어떤 작업에 시간과 자원이 쓰이는지 측정하는 과정입니다. GPU Profiler는 GPU 작업을 이벤트와 패스 단위로 살피게 해 줍니다. 이벤트는 도구가 이름을 붙여 기록한 작업 구간입니다. 여러 프레임의 경향과 한 프레임의 세부 구조를 함께 확인합니다.

작동 과정

stat gpu로 큰 작업 그룹을 확인하고 profilegpu 또는 Unreal Insights의 GPU 시간축으로 원인을 좁힙니다. 부모 이벤트 안에 자식 이벤트가 포함될 수 있으므로 모두 더하면 중복 계산할 수 있습니다. 서로 다른 큐에서 겹쳐 실행되는 작업도 있어 각 항목의 합과 전체 프레임이 다를 수 있습니다. 도구 버전 차이는 보충 주석 4를 참고합니다.

아트 실무

같은 골목에서 데칼 투영 범위만 줄여 봅니다. 데칼 관련 시간과 전체 GPU 시간이 같이 줄었는지, 얼룩 범위와 반사가 유지됐는지 확인합니다. 다른 패스가 늘었다면 비용이 이동한 것일 수 있습니다. 장면 전체가 CPU에 의해 제한되고 있다면 GPU 개선이 곧 FPS 개선으로 나타나지 않을 수도 있습니다.

실무에서 비교하고 검증하기

성능 비용

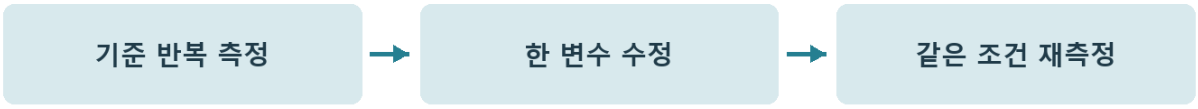
에디터와 캡처 도구도 자원을 사용합니다. 셰이더 준비와 데이터 로딩이 끝나 안정된 상태로 만드는 과정을 워밍업이라고 합니다. 첫 진입의 준비 비용은 별도 기록하고, 안정 구간의 전후 비교와 섞지 않습니다. 스크린샷 저장 순간은 평상시의 프레임 시간 측정으로 사용하지 않습니다.

언리얼 실습

기기·엔진 버전·그래픽 API·해상도·품질 설정과 카메라 경로를 기록합니다. 같은 길이로 세 번씩 실행해 Frame 중앙값·P95, GPU 중앙값과 해당 패스 시간을 남깁니다. 중앙값은 측정값을 크기순으로 놓았을 때 가운데 값입니다. 이동 영상도 비교한 뒤 변경을 되돌려 효과가 재현되는지 확인합니다.

핵심 요약

한 변수의 수정이 특정 패스, 전체 프레임과 화질에 미친 영향을 같은 조건의 기록으로 설명합니다.



판정 = 총 프레임 시간 + 패스 변화 + 이동 화질

직접 제작 설명도 / 같은 카메라·조건에서 기준 측정, 한 변수 수정, 재측정을 반복합니다. 패스 시간 감소와 총 프레임 개선·화질 유지를 함께 확인합니다.

공식 자료와 이어 읽기

[03] Introduction to Performance Profiling and Configuration

[28] Stat Commands

[29] Beyond Print String · GPU Profiler

CH25 CH01 용어집 보충 주석

골목 씬의 성능 분석

재현할 장면 정하기

모퉁이를 도는 30초 카메라 경로를 저장합니다. 데칼·가로등·유리·풀을 포함하고 기기, 드라이버, 엔진 버전, 그래픽 API, 빌드 구성과 화면 크기를 기록합니다. 프레임 제한과 VSync를 진단 조건에 맞춰 통제하고 제품 설정에서도 다시 확인합니다.

처음 들어갈 때만 끊기는가

첫 진입과 다시 방문했을 때를 비교합니다. 처음에만 반복되면 텍스처·메시를 읽는 작업과 실행할 셰이더·그리기 상태를 준비하는 작업을 조사합니다. 항상 같은 구도에서 GPU 시간이 길면 해당 프레임의 무거운 패스를 찾습니다.

그림자를 갱신하는가

풀밭을 볼 때 Shadow Depths가 길면 VSM의 페이지 시각화를 봅니다. 먼 풀의 WPO만 끈 비교와 그림자만 바꾼 비교를 따로 합니다. 그림자의 움직임과 외관, 페이지 갱신과 GPU 시간을 함께 기록합니다.

결과 남기기

변경 내용, 카메라 구간, 반복 횟수, Frame 중앙값·P95, Game·Draw·GPU 시간, 바꾼 패스 시간, 메모리 사용과 이동 중 외관 차이를 남깁니다. 평균이 좋아도 특정 구간이 끊기면 그 프레임의 CPU·GPU·대기 구간을 다시 봅니다.

효과 판단하기

특정 패스가 줄어도 전체 프레임이 그대로면 다른 병목이 남았을 수 있습니다. 전체 시간과 화질이 목표에 맞는지 확인하고, 변경을 되돌렸다가 다시 적용해 결과가 재현되는지 검증합니다.

용어집

뜻을 확인한 뒤 표시한 챕터의 원리와 사례로 돌아가세요.

CPU와 GPU

CPU는 게임 상태와 작업 준비 등을 처리하는 범용 프로세서, GPU는 많은 그래픽 계산을 병렬로 처리하는 프로세서입니다. [CH01·25]

스레드

프로그램의 명령을 이어 실행하는 흐름입니다. 렌더 스레드가 CPU에서 일한다는 점과 GPU의 실행을 구분합니다. [CH01·25]

렌더 스레드

주로 CPU에서 장면의 가시성과 그릴 명령을 준비하는 실행 흐름입니다. [CH01·25]

RHI와 그래픽 API

RHI는 엔진의 그래픽 명령을 플랫폼별 그래픽 API에 연결하는 계층입니다. API는 기능을 사용하기 위해 정한 요청 방식입니다. [CH01·25]

프레임과 렌더 패스

프레임은 화면 한 장을 만드는 처리 구간이며, 패스는 그 안에서 특정 입력과 결과를 다루는 작업 단위입니다. [CH01·26]

지오메트리와 메시

지오메트리는 형태와 위치에 관한 기하 정보입니다. 이 문서의 메시는 주로 삼각형으로 구성한 3D 표면 형상입니다. [CH01·17]

정점

삼각형의 꼭짓점입니다. 위치 외에 표면 방향과 텍스처 좌표 같은 정보도 가질 수 있습니다. [CH01·10]

좌표 변환과 투영

모델 내부의 위치를 장면·카메라 기준으로 바꾸고 화면 위치에 대응시키는 과정입니다. [CH01]

셰이더

GPU에서 실행되는 계산 프로그램입니다. 정점과 화면의 표면 값 등을 처리하는 여러 종류가 있습니다. [CH01·10]

래스터화

화면에 투영한 삼각형이 어느 화면 위치를 덮는지 판정하는 과정입니다. [CH01]

용어집 계속

뜻을 확인한 뒤 표시한 챕터의 원리와 사례로 돌아가세요.

샘플과 프래그먼트

래스터 문맥의 샘플은 커버리지를 검사하는 화면 위치입니다. 프래그먼트는 삼각형에서 생긴 화면 위치별 처리 후보입니다. [CH01·05]

보간

알고 있는 값 사이에서 중간 위치의 값을 계산하는 것입니다. 삼각형 내부의 UV 등을 구하는 데 쓰입니다. [CH01]

UV와 텍스처 샘플

UV는 표면을 이미지의 위치에 대응시키는 2D 좌표입니다. 텍스처 샘플은 그 좌표에서 이미지 값을 읽는 작업입니다. [CH01·19]

법선과 노멀 맵

법선은 표면 방향입니다. 노멀 맵은 작은 표면 방향 변화를 이미지로 표현해 빛 반응에 사용합니다. [CH03·07]

탄젠트

표면을 따라 놓인 방향 기준입니다. 탄젠트 공간 노멀 맵은 이 로컬 방향 기준을 이용해 표면 방향 변화를 표현합니다. [CH03]

버퍼와 렌더 타깃

버퍼는 계산 데이터를 보관하는 메모리 공간입니다. 렌더 타깃은 그리기 결과를 기록할 이미지 자원입니다. [CH01·03]

GBuffer와 Base Pass

GBuffer는 조명에 필요한 표면 정보 묶음입니다. Base Pass는 주로 불투명 표면 정보를 준비하는 작업입니다. [CH03]

DBuffer와 데칼

데칼은 다른 표면에 투영하는 얼룩·표식 등의 표현입니다. DBuffer는 Base Pass 전에 그 표면 수정 정보를 준비합니다. [CH04]

깊이 버퍼와 Early Z

깊이 버퍼는 앞쪽 표면 깊이를 저장합니다. Early Z는 가능한 경우 이 정보로 가려진 후보를 일찍 제외합니다. [CH05]

Opaque와 Masked와 Translucent

각각 불투명, 위치별로 남길지 버리는 구멍 마스크, 뒤의 색과 합성하는 반투명 표면을 가리킵니다. [CH06·12]

용어집 계속

뜻을 확인한 뒤 표시한 챕터의 원리와 사례로 돌아가세요.

Overdraw와 커버리지

Overdraw는 같은 화면 위치의 반복 처리입니다. 커버리지는 도형이 화면의 어느 범위를 덮는지를 뜻합니다. [CH06]

PBR

재료와 빛의 상호작용을 물리적으로 그럴듯하게 근사하는 제작 방식입니다. [CH07]

Base Color와 Metallic과 Roughness

재질의 기본 색, 금속성, 반사가 퍼지는 정도를 제어하는 입력입니다. 최종 화면 색과 역할이 다릅니다. [CH07]

Material Instance와 파라미터

인스턴스는 원본 재질 구조를 공유하는 설정값 변형입니다. 파라미터는 바꿀 수 있도록 노출한 입력값입니다. [CH07·10]

BRDF와 BSDF

BRDF는 방향에 따른 반사 반응을 기술합니다. BSDF는 표면을 통한 투과까지 포괄하는 더 넓은 개념입니다. [CH08]

Fresnel과 F0

Fresnel은 각도에 따른 계면 반사율의 변화입니다. F0는 법선 방향 입사의 반사율입니다. [CH09]

컴파일과 셰이더 순열

컴파일은 실행할 프로그램을 준비하는 과정입니다. 순열은 기능·정적 설정 등의 조합에 따른 실행 버전을 뜻합니다. [CH10]

Static Switch와 WPO

Static Switch는 컴파일할 때 경로를 고릅니다. WPO는 정점의 월드 위치에 변화를 더하는 재질 입력입니다. [CH10]

캐시와 대역폭

캐시는 데이터를 다시 읽기 쉽게 두는 임시 저장 영역입니다. 대역폭은 일정 시간에 옮길 수 있는 데이터 양입니다. [CH03·10]

Slab과 Closure

Slab은 Substrate의 재료 정의 단위입니다. Closure는 실행 시 조명에 대해 평가할 재료 반응의 단위를 가리킵니다. [CH11]

용어집 계속

뜻을 확인한 뒤 표시한 챕터의 원리와 사례로 돌아가세요.

파라미터 블렌딩

복수 재료의 설정값을 하나의 재료 반응으로 합쳐 근사하는 방식입니다. [CH11]

알파 합성과 굴절

알파 합성은 앞뒤 색을 비중에 따라 섞는 계산입니다. 굴절은 물질 경계에서 빛 방향이 바뀌는 현상입니다. [CH12]

SSS와 MFP

SSS는 표면 아래 빛의 산란입니다. MFP는 내부 상호작용 사이의 평균 거리 스케일을 가리킵니다. [CH13]

Actor와 컴포넌트

Actor는 레벨에 배치하는 오브젝트 단위입니다. 컴포넌트는 그 안의 메시·조명 등 기능 요소입니다. [CH14·15]

Draw Call과 메시 섹션

Draw Call은 GPU에 그리기를 제출하는 명령입니다. 섹션은 보통 재질을 따로 적용할 수 있는 메시의 구역입니다. [CH14]

배칭과 인스턴싱

배칭은 호환되는 작업을 묶는 접근입니다. 인스턴싱은 메시 등의 데이터를 공유해 반복 배치를 처리하는 방식입니다. [CH14·15]

ISM과 HISM

반복 Static Mesh를 컴포넌트 안에서 관리하는 기능입니다. HISM은 공간적으로 묶은 계층을 활용합니다. [CH15]

LOD와 FOV

LOD는 필요한 화면 디테일에 맞춘 대체 표현입니다. FOV는 카메라가 담는 시야각입니다. [CH16]

클러스터와 폴백 메시

Nanite 클러스터는 작은 삼각형 묶음입니다. 폴백 메시는 다른 기능이나 조건에서 사용하는 대체 표현입니다. [CH17]

컬링과 bounds

컬링은 불필요한 가시 후보를 제외하는 과정입니다. bounds는 이런 판정 등에 쓰는 물체를 둘러싼 범위입니다. [CH18]

용어집 계속

뜻을 확인한 뒤 표시한 챕터의 원리와 사례로 돌아가세요.

밉맵과 필터링

밉맵은 축소 이미지의 계층입니다. 필터링은 필요한 범위의 값을 읽고 섞어 화면에 대응시키는 처리입니다. [CH19]

스트리밍 풀과 I/O

풀은 관리할 메모리 예산의 한 묶음입니다. I/O는 디스크 등에서 데이터를 읽고 옮기는 입출력 작업입니다. [CH20]

Screen Trace와 거리장

Screen Trace는 화면 정보로 경로를 조사합니다. 거리장은 표면까지의 거리를 저장한 장면 표현입니다. [CH21]

Surface Cache와 Cards

Surface Cache는 표면 정보를 빠르게 조회하는 체계입니다. Cards는 표면을 여러 방향에서 담는 표현에 사용됩니다. [CH21]

레이 트레이싱과 가속 구조

광선과 물체의 교차를 추적하는 계산과, 교차 후보를 빨리 찾도록 준비한 자료 구조입니다. [CH21]

VSM 페이지와 캐스터

페이지는 가상 그림자 맵의 작은 영역입니다. 캐스터는 그림자를 만드는 물체입니다. [CH22]

캐시 무효화

예전에 계산해 둔 결과가 현재 장면에 맞지 않아 다시 계산해야 하는 상태입니다. [CH22]

히스토리와 지터

히스토리는 이전 프레임의 저장 정보입니다. 지터는 프레임마다 샘플 위치를 조금씩 바꾸는 동작입니다. [CH23]

모션 벡터와 재투영

모션 벡터는 화면상 이동 정보입니다. 재투영은 과거 정보를 현재 화면 위치에 대응시키는 과정입니다. [CH23]

고스팅

과거 정보가 잘못 남아 움직이는 표면 뒤로 끌리거나 번져 보이는 시간적 아티팩트입니다. [CH23]

용어집 계속

뜻을 확인한 뒤 표시한 챕터의 원리와 사례로 돌아가세요.

내부 해상도와 업스케일러

내부 해상도는 주요 장면 계산의 크기입니다. 업스케일러는 그 결과 등을 이용해 더 큰 출력 화면을 만드는 기능입니다. [CH24]

병목과 VSync

병목은 처리율을 제한하는 원인입니다. VSync는 디스플레이 갱신에 출력 시점을 맞추는 기능입니다. [CH25]

중앙값과 P95

중앙값은 크기순 가운데 값입니다. P95는 측정값의 95%가 그 이하에 놓이는 경계로, 느린 프레임을 살필 때 사용합니다. [CH25·26]

프로파일링과 워밍업

프로파일링은 실행 중의 비용 측정입니다. 워밍업은 셰이더 준비·로딩 등을 끝내 안정된 비교 조건을 만드는 과정입니다. [CH26]

톤매핑과 Bloom

톤매핑은 장면 밝기를 표시할 범위로 옮기는 처리입니다. Bloom은 밝은 부분이 주변으로 번지는 화면 효과입니다. [CH01]

Scalability와 쿠킹

Scalability는 엔진의 품질 설정 묶음입니다. 쿠킹은 대상 플랫폼에서 실행할 배포용 데이터를 준비하는 과정입니다. [CH26]

보충 주석

본문의 흐름을 이해한 뒤 구현과 적용 조건을 더 확인할 때 읽습니다.

1 스레드와 파이프라인 도식의 범위

RHI는 인터페이스 계층의 이름입니다. RHI 작업이 별도 스레드에서 처리되는 방식과 병렬화 구성은 플랫폼·설정에 따라 달라집니다. 본문의 단계 그림은 작업 의존 관계를 이해하기 위한 것입니다. 실제 GPU 프레임은 그림자·추적·후처리 등의 분기와 동시 실행을 포함하므로 GPU 캡처에서 확인합니다.

2 BRDF 수식을 더 읽을 때

대표적인 미세면 스펙큘러 항은 D×F×G를 4(N·L)(N·V)로 나눈 형태로 설명할 수 있습니다. D는 미세면 방향 분포, F는 Fresnel 반사율, G는 미세면 사이의 가림입니다. N은 표면 법선, L은 빛 방향, V는 시선 방향이며 점 기호는 두 방향의 관계를 계산하는 내적입니다. 실제 UE 재질의 모든 경로가 이 한 식과 같은 구현이라는 뜻은 아닙니다.

3 Substrate 저장 형식과 버전

이 문서는 UE 5.8 문서와 실습 프로젝트의 5.8.4를 기준으로 합니다. Adaptive GBuffer의 지원 플랫폼과 단순화 조건, DBuffer 요구 사항은 해당 버전의 공식 문서에서 확인합니다. 60Hz 지향은 설계 목표이며 실제 게임의 60 FPS 달성을 보장하는 수치가 아닙니다. 프로젝트 범위 변경은 재시작이나 셰이더 재컴파일을 요구할 수 있습니다.

4 GPU 프로파일러의 표시와 시간 합계

UE 5.6 이후 구성에서는 구형 GPU Visualizer 팝업과 같은 표시를 전제로 하지 않고 로그와 Unreal Insights의 GPU 트랙을 확인합니다. 중첩 이벤트, 동시 실행과 CPU·GPU의 대기 때문에 항목의 단순 합이 총 프레임과 다를 수 있습니다. 공식 도구 설명과 대상 빌드의 실제 동작을 함께 확인합니다.

출처 목록

Epic Games와 그래픽 API 공식 문서 및 원 저자의 발표

01 Rendering Settings

02 Render Dependency Graph

03 Introduction to Performance Profiling and Configuration

04 Forward Shading Renderer

05 Programming with Substrate GBuffer Formats

06 Decal Materials

07 Viewport Modes

08 Physically Based Materials

09 Using Fresnel

10 Overview of Substrate Materials

11 Material Editor UI

12 Material Inputs

13 Lit Translucency

14 Using Refraction

15 Subsurface Profile Shading Model

16 Mesh Drawing Pipeline

17 Instanced Static Mesh Component

18 Optimizing LOD Screen Size Per Platform

출처 목록 계속

Epic Games와 그래픽 API 공식 문서 및 원 저자의 발표

19 Nanite Virtualized Geometry

20 Texture Asset Editor

21 Texture Streaming Overview

22 Texture Streaming Metrics

23 Lumen Technical Details

24 Lumen Performance Guide

25 Virtual Shadow Maps

26 Anti-Aliasing and Upscaling

27 Temporal Super Resolution

28 Stat Commands

29 Beyond Print String · GPU Profiler

30 Visibility and Occlusion Culling

31 Real Shading in Unreal Engine 4 · Brian Karis

32 Scripting the Editor Using Python

33 Parallel Rendering Overview

34 Direct3D Rasterizer Stage

35 Material Instances

36 Cornellbox pathtracing · Trevorgoodchild · Public domain

출처 목록 계속

Epic Games와 그래픽 API 공식 문서 및 원 저자의 발표

37 LearnOpenGL Coordinate Systems · 이미지 출처

38 LearnOpenGL Anti Aliasing · 이미지 출처

39 LearnOpenGL Deferred Shading · 이미지 출처

40 LearnOpenGL PBR Lighting · 이미지 출처

41 LearnOpenGL PBR Theory · 이미지 출처

42 LOD Example · Sirotk · CC BY-SA 3.0

43 Mipmap Aliasing Comparison · BillyBob CornCob · CC0

44 LearnOpenGL 이미지 이용 조건 · CC BY 4.0

45 Epic Nanite Preserve Area 비교 이미지 · UE 5.5